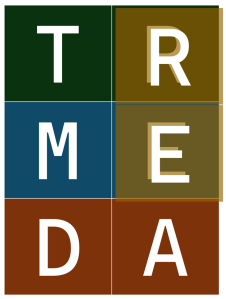

Building the TReMeDa Datasets

A Research Compendium

Building the TReMeDa Datasets
A Research Compendium

Chris Moreh
Version: Development draft
Produced: 2026-08-24

Site repository: https://github.com/CGMoreh/tremeda-compendium-site
Source code: https://github.com/CGMoreh/tremeda-compendium (Private repo – request access to contribute)

Contents

Introduction

The TReMeDa project

The Trust Research Methodology Database (TReMeDa) is building a curated database of metadata and associated replication data for English-language publications on the topic of 'social trust' in the social sciences, broadly construed, over the three decades since 1990.

"Social trust" is treated here as a broad concept that spans many definitions and operationalisations of trust in a social-science context – not only generalised trust, but interpersonal and institutional trust as well. That breadth is deliberate, and it is also the central methodological difficulty: a publication may carry an important trust component without naming it in its title or abstract, and the bare word "trust" is often used as shorthand whose precise sense is supplied by the journal or discipline. The most encompassing discovery strategy is therefore to capture everything that mentions trust in title, abstract or keywords, and then use further evidence – textual, publication-level and disciplinary – to decide whether a given output really concerns the kind of trust the project is about.

The three datasets

The database is organised as three nested subsets, each serving different aims.

THEME – the thematic dataset. The master layer: general bibliometric metadata for trust research in the social sciences, drawn from Web of Science, Scopus, and OpenAlex, and augmented with classifications that can be extracted at acceptable reliability by machine-learning tools (for example a broad methodological split into theoretical, empirical-qualitative, and empirical-quantitative work). Raw size as assembled: 75,302 entries, manual additions and backfills included, before exclusions due to misidentification on technical grounds (i.e. not quality-driven).

META – the meta-analytical dataset. A restriction of THEME to primary quantitative empirical studies, augmented with deeper descriptors: primary versus secondary data sources, whether trust is the dependent or independent variable, reproducibility and data-access statements, statistical methods, and causal claims. Scale: the candidate pool carried forward from screening holds 11,654 publications coded as in scope and as quantitative or mixed-methods, drawn from the portion of the corpus screened so far. How many of those become META records depends on how many full texts can be obtained and read, so the extracted set will be substantially smaller than the pool.

METHOD – the methodological dataset. The smallest and most detailed layer: studies whose findings can be replicated from publicly available data or author-provided replication packages, accompanied by documented analytical datasets and annotated code. Its purpose is pedagogical – to underpin a textbook on quantitative sociological methodology applied to researching trust, and to serve instructors more generally. Expected scale: tens of publications rather than hundreds, spanning a broad range of disciplines and methods, and broken down into a larger number of analytical datasets designed to demonstrate individual methods and/or published outputs. The set is small by design, because every entry costs the work of sourcing the data, reproducing a published result well enough to trust the entry, and documenting the transformation chain.

What these notebooks document

These notebooks are the running, cleaned record of the methodological work behind the three datasets and of the decisions that work supports. They are organised by the same three layers. Work so far has concentrated on **THEME**, and the notebooks in that part document the comparison of bibliographic databases that settled the discovery design, the record-level trust classifier, the assembled corpus, the pipeline that builds it, the codebook for every column, and the screening programme that codes each record for scope and study type.

Screening has run to completion over the records whose abstracts carry an explicit open licence, under a frozen codebook and with reliability measured against a blind second pass. The rest of the corpus is queued for an open-weight model running on university infrastructure, and the records that hold no

abstract at any index are coded from title and metadata alone. **META** has its candidate pool from the completed screening and an extraction stage still being specified; its notebooks will follow as that stage is built, and **METHOD** remains a specification.

I THEME – the thematic dataset

Overview

The thematic dataset is the foundation the other two are carved from, so the first methodological task is to settle how it is discovered – which sources to search, and how. The project does not begin by assuming that any one database is sufficient; it begins from what is already known.

A background notebook (Notebook 1.) reviews what the literature and OpenAlex's own documentation show about the bibliographic databases available for this work: that abstract metadata is being withdrawn from open databases at scale, and that OpenAlex's subject labels are an automated, AI-built artefact rather than an intrinsic property of a paper. That context sets the terms for a practical comparison.

Which sources, and can one suffice?

The conventional approach combines several subscription databases – expensive to maintain and awkward to keep current. OpenAlex is an attractive single source – open, free, very large, and queryable by API – but only if it does not *miss* what the curated databases (Scopus, Web of Science) would catch. The way to find out is to compare them directly. A focused single-year (2015) audit did this across the three most authoritative bibliographic databases the team has institutional access to, verifying every apparent gap against the live databases to separate a genuine coverage gap from a mere retrieval gap; it is summarised in section 3.1.

The audit reaches a layered answer. The OpenAlex *database contains* almost everything the curated searches return, so coverage is not the obstacle; but no single query *retrieves* it all, because each database draws its disciplinary and document-type boundaries differently. Discovery therefore has to draw on all three sources together, merged by identifier and filtered downstream.

The databases notebook also sets out the **selection design** built on that answer – the curated Scopus and Web of Science keyword searches, the OpenAlex twelve-topic union, and the by-DOI backfill that recovers curated metadata for the only-OpenAlex works (section 1.4, section 1.5). A further notebook describes the record-level **trust classifier** that triages each retrieved record by the sense in which it uses the word (Notebook 2.); the assembled **2000–2025 corpus** is reported and interrogated in Notebook 3.; the construction **pipeline** is summarised visually in Notebook 4.; and every column of the dataset is catalogued in the **codebook** (Notebook 5.).

The licence warrant on each abstract

Screening a record means reading its abstract, and reading it by machine means deciding where that text may be sent. The project's answer applies per article. The presence of an abstract in a database is evidence about held text and says nothing about rights, so the warrant to transmit that text to an external service has to come from the work's own licence, which `OA_licence` records. The derived flag `abstract_externally_sendable` is true only where a usable abstract is held and that licence carries an explicit open slug. The warrant sorts into three tiers: 15,393 works under a strict open licence, 2,824 non-commercial and 3,276 no-derivatives. An `other-oa` designation, which asserts open availability without naming terms, does not clear the bar, and the abstracts recovered by the curated backfill stay outside it altogether – used for local processing, never transmitted (section 1.5).

That rule, with a slightly stricter length floor applied when a record is prepared for coding, partitions the 71,108 works in the analysis window three ways. **21,493** carry a warrant and may be coded by an external service. **47,097** hold a usable abstract with no warrant, and are reserved for coding on the university's own hardware. **2,518** hold no abstract at any index the project can reach. A retrieval sweep across Semantic Scholar, Crossref and Europe PMC, which sent out DOIs and nothing else, folded 2,843 usable abstracts into the corpus – 2,766 of them replacing local texts that failed the quality checks and 77 filling records that had none – and API retrieval is exhausted for the residue that remains.

Screening the corpus

Screening asks two questions of every record: whether it concerns social trust, and what kind of study it is. The answers are held as `h_class`, which separates central from edge-case, out-of-scope, undetermined and fake records, and as `h_methodology`, which separates theoretical, qualitative, quantitative, mixed and review work from the two codes that record what a record fails to establish. This is a human codebook, frozen at version 3 after three calibration waves, and its two variables sit in the data beside the rule classifier's `trust_scope` and `trust_sense` rather than replacing them – the automatic triage is a prior on the coding, not a substitute for it.

The externally warranted track was coded in August 2026: all 21,493 records under the frozen codebook. A blind second pass over 1,250 seeded records, whose coders were forbidden to read the first pass, measures how far that coding reproduces – Cohen's kappa 0.858 on `h_class`, 0.885 on `h_methodology`, and 0.923 on the collapsed quantitative-or-mixed decision that admits a record to META. The decision the funnel depends on is thus the most reliable one in the coding. What class disagreement remains sits almost entirely on the central/edge-case boundary, which a downstream filter can revisit.

The 2,518 records without an abstract are coded from title and metadata alone, under the codebook's thin-metadata rules, and their labels are weak by design: a code that requires a stated design is close to unreachable when no abstract states one. They are kept as a separate dataset and never merged into the abstract-based one, so the reliability figures above describe only the coding they were measured on.

Three datasets exist on disk as a result. `theme_claude_prod` holds the 21,493 externally coded records; `theme_titleonly_prod` holds the 2,518 title-coded ones; and `meta_raw` holds the 11,654 records that clear the screening gates and are quantitative or mixed – the candidate pool for full-text extraction, built so far from the externally coded track alone and due to grow when the local track lands.

Open questions

The local-model run. The 47,097 records that hold a usable abstract without a warrant are to be coded by an open-weight model on university high-performance computing, which keeps licensed text on the institution's own machines. The handover kit is built: the condensed codebook as a system prompt, worked exemplars, a schema for constrained decoding, training splits that reserve the blind-pass records for evaluation, and a scoring harness that reproduces the project's own kappa figures before it is asked to score anything new. Its acceptance gates sit just below the reference coding's measured self-agreement, since a gate set above that would fail even a perfect copy of the coder being imitated. Whether a 32B-class model clears them is not yet known, and the union of the two coded tracks waits on the answer.

What META extracts, and from how many papers. The candidate pool of 11,654 is far larger than the few hundred to two thousand publications the original design anticipated for META, and two things decide what the extraction stage actually runs on. The first is the list of fields to extract, which locks the schema the full-text stage is built against. The second is how far full-text availability and any further selection criteria prune the pool. A GROBID pilot has already settled the parsing question, so these are the remaining constraints on the stage.

Where the title-coded records go. Each of the 2,518 needs routing either to full-text review or to documented exclusion, and three candidate tests have been measured against the coded labels. The existing quantitative-or-mixed gate selects 7 records; admitting `empirical-uncertain`, on the reasoning that full text is what would resolve it, selects 578; taking everything not positively excluded selects 1,484. The choice trades reviewing effort against the recall that weak labels cost.

Two extensions to discovery are still to be built. The first is a disciplinary-core test: a curated search restricted to the sociology and political science core, run over the full span and matched against the assembled corpus, to establish how much of that core the multi-source design actually holds. The second is a citation reference miner, which would work the OpenAlex citation graph the pipeline already stores to reach studies that engage the trust literature without carrying a trust term in any field a keyword search can read. Both build on the coverage and added-value evidence that produced the selection design, and both sit behind the screening and extraction work in the project's order.

1. Bibliographic databases: Web of Science, Scopus, OpenAlex

> **i** Status: background + the implemented selection design
>
> This notebook does two things. First it sets out what is already known – from the literature and from each provider's own documentation – about the three bibliographic databases this project relies on: their coverage, their search tools, and the keyword and classification systems they add. Then it sets out the **selection design** the project actually ran on them, and the curated by-DOI backfill that completes it. The empirical corpus that results is the subject of Notebook 3..

The thematic dataset is discovered from three bibliographic databases – **Web of Science**, **Scopus** and **OpenAlex** – chosen because they are the three most authoritative sources the team has institutional access to, and because they are complementary rather than interchangeable. This notebook first describes what each one is and what it adds beyond a list of references (its search tools and its keyword/classification systems), then turns to the issue that shapes every later design choice – the metadata we query, the abstract above all, is not stable – and finally sets out how the project searches all three and reconciles the result.

1.1 What the three databases are, and what they add

All three index scholarly works and let us search them, but they are built differently, cover the literature differently, and – crucially for a keyword project – attach different *added* metadata to each record. The differences are not incidental; they are the reason no single source suffices.

1.1.1 Web of Science

Web of Science (Clarivate) is the oldest and the most **selective** of the three: its Core Collection is an editorially curated set of journals chosen for citation impact, so it is comparatively clean but narrower, and its coverage skews towards established English-language journals. For discovery, its `TS=` ("topic") field reaches four fields at once – title, abstract, **author keywords**, and **Keywords Plus**. Keywords Plus is the distinctive addition: index terms generated **algorithmically by Clarivate from the titles of the work's cited references**, not supplied by the author. Because they are derived from what a paper *cites* rather than what it *says*, Keywords Plus can surface a study on a term it never uses itself – a quantitative trust paper that cites the trust canon heavily can be retrieved on "trust" even when its own title and abstract never mention it. Web of Science also assigns each work the subject **Categories** and **Research Areas** of its journal – a journal-level classification, not a per-article one.

1.1.2 Scopus

Scopus (Elsevier) is **broader**: it indexes more journals, conference proceedings and book series than Web of Science, so it returns more, at some cost in selectivity. Its `TITLE-ABS-KEY` field reaches title, abstract, **author keywords** and **index keywords**. The index keywords are Scopus's distinctive addition: **controlled-vocabulary terms assigned from standardised thesauri** (for example Emtree for the biomedical literature) rather than chosen by the author – a curator/algorithm-assigned layer that normalises terminology across a field. Scopus classifies each source under the **All Science Journal Classification** (ASJC: 27 subject areas, ~300 minor categories) – again a journal-level scheme. The practical effect is that Scopus is the broadest keyword net of the three and the one most likely to return the cross-disciplinary tail.

1.1.3 OpenAlex

OpenAlex (Priem Et al., 2022) is the newcomer and the structural departure: a **fully open**, free, very large index (over 250 million works) launched in 2022 by the OurResearch non-profit as a successor to the discontinued Microsoft Academic Graph, built on open data, an open API and open-source code. Two features make it attractive as a backbone. It is open and queryable at scale without a subscription, so it can serve as the discovery substrate and citation graph for a project of this size. And rather than rely only on journal-level subject categories, it assigns **each work its own machine-assigned topic** from a citation-built hierarchy – the genuinely novel, LLM-infused addition that the next section unpacks, because three of this project's design choices turn on how much weight those topic labels can bear. The trade-off is that OpenAlex is the least selective and, as the rest of this notebook shows, the most exposed to the metadata instability that now threatens keyword discovery.

1.2 How OpenAlex builds its topics, and what it offers

Because the OpenAlex topic system is both the reason to use it and a thing to be cautious about, it is worth setting out exactly how those labels are made. The current scheme, **Topics**, is documented in the CWTS team's open whitepaper and dataset (Eck, 2024; Eck & Waltman, 2024) and in OpenAlex's own end-to-end process notes: a four-step pipeline whose every label is the output of stacked automated layers, none of them systematically audited.

1. **Cluster.** CWTS clustered the ~71 million OpenAlex works (2000–2023) that carry incoming or outgoing citations – linked by ~1.7 billion citation edges – using the extended direct-citation approach and the Leiden algorithm (Waltman & Eck, 2012; Traag Et al., 2019). This produced **4,521 micro-level** research areas, grouped into **917 meso** and **20 macro** areas. The clusters are defined purely by citation structure: no text, no keywords.
2. **Label.** To name 4,521 clusters without manual effort, CWTS fed the **titles of a sample of each cluster's most-cited papers** to a large language model (an updated **GPT-3.5 Turbo**; the open labelling tool also supports GPT-4) which returned, per cluster, a short label, a long label, ten keywords, a summary and a Wikipedia URL (Eck, 2024). The authors note that they did not systematically evaluate label quality and that the same approach failed at higher aggregation levels. The labels, keywords and summaries in the mapping table are this model's output.
3. **Map to a hierarchy.** OpenAlex then placed each citation-defined topic into the Scopus **ASJC** structure (4 domains, 26 fields, 254 subfields) that it exposes as `domain` / `field` / `subfield`, using a *second* LLM pass, because asking a person to place over 4,000 topics by hand was impractical. So the `domain` we filter on is not an intrinsic property of a paper – it is an LLM's judgement about which ASJC bin a citation cluster's machine-written label most resembles, and the team flags that some topics could sit in several subfields.
4. **Classify works.** Finally, to tag the ~250 million works – only a third of which carry citation data and only about half a usable abstract – OpenAlex trained a supervised model on the CWTS labels: a fine-tuned multilingual BERT over the merged title-and-abstract text, fed into a network that also takes a journal-name embedding and "gold citation" features. The model was trained with features randomly held out so it degrades gracefully when the abstract or the citations are missing. The per-work classifier is therefore *not* an LLM: the only LLM steps write the bin names (step 2) and file the bins into the hierarchy (step 3).

Two properties matter for us. First, the per-work label is **single** (`primary_topic`) and is assigned from four signals of uneven reliability – title (~90% of works), abstract (~50%), journal name, and citations (~33%). Second, because the model was built to tolerate a missing abstract, abstract withdrawal does not break topic assignment outright; it removes the richest *text* signal and pushes the model onto title, journal and citation features. That fallback is adequate for placing a paper in broad subject space, but it is precisely the signal that cannot separate the *senses* of trust: a closed-access paper titled "Antecedents of relationship commitment," stripped of its abstract, is placed by its journal and its citations, not by whether its trust is interpersonal, institutional or brand trust.

1.2.1 Where "trust" lives in the OpenAlex topic map

The mapping table also settles a question we had set aside: is there a single natural "trust topic" to scope on? There is not. Searching all topics' labels, keywords and summaries for trust returns **27 topics**, spread across three domains (21 Social Sciences, 5 Physical Sciences, 1 Health Sciences) and many fields. Only one

carries "trust" in its actual label – topic 10927, "Access Control and Trust" – and it is an online-reputation and computer-security cluster filed, through the step-3 mis-mapping above, under the Sociology and Political Science subfield: a clean instance of exactly the off-target trust this project wants to exclude. The trust the project *does* want is scattered across a handful of topics in different fields, several of which do not announce trust in their label at all (social capital, experimental behavioural economics, risk perception, e-government, policing, vaccine hesitancy, oxytocin-and-trust, culture-and-development). No single-topic scope can work, and scoping by topic *label* or *subfield name* is unsafe; the defensible middle path is a **curated union of inspected topics**, which is exactly what the selection design (section 1.4) uses.

Set against its construction, the OpenAlex topic system offers a great deal as **input** and very little as **verdict**. As input it is excellent: a free, citation-grounded clustering of the whole literature, with per-work labels robust to missing metadata. As a verdict it is weak for our purpose: single-label, three automated layers deep with no systematic audit, its domain placement an LLM's guess about ASJC bins, and its discriminating text signal the very abstract now being withdrawn. The reasonable use is to lean on it for recall and structure and to reserve the in/out and trust-sense judgements for our own content-level screen (Notebook 2.). Comparative work on OpenAlex coverage and metadata completeness reaches the same posture – strong as an open substrate, uneven enough to verify rather than trust blindly (Alonso-Álvarez & Eck, 2025).

1.3 Why we cannot rely on OpenAlex alone

A discovery design that leans on OpenAlex uses the abstract in three places at once: the keyword clause that finds candidate papers, any downstream abstract-level screen, and OpenAlex's own abstract-fed topic labels. Anything that degrades the abstract degrades all three at once – and the degradation is not uniform: it falls hardest on closed-access, large-publisher, recent work, disproportionately the high-quality quantitative studies the project most wants to keep.

Abstracts are being withdrawn, and the incentive is AI. Aaron Tay's (2025) analysis frames open abstracts as "the petrol tank for AI discovery" and measures the drain: Elsevier abstract coverage in OpenAlex fell from roughly 82% in November 2024 to 22.5% by September 2025, Springer Nature from about 50% to 35.8%. The OpenAlex maintainers describe the same process from inside – many abstracts were inherited from Microsoft Academic Graph without an open licence and have been removed on copyright grounds (Springer Nature, then ACS, OUP, Wiley, Cell Press, Elsevier and others). Now that AI companies will pay for training text, publishers have re-classified the abstract as a strategic asset and are closing the indirect channels open aggregators relied on, so the withdrawal will not simply reverse. Stansfield et al. (2025), testing OpenAlex for systematic reviews, found previously retrievable studies became unfindable once their abstracts were removed, with recall drifting downward as the corpus changed underneath them.

The open-abstract commons was never full. The Initiative for Open Abstracts [I4OA; Initiative for Open Abstracts (2020)] asks publishers to deposit abstracts openly in Crossref; it succeeded with many smaller and fully-open publishers but not with the largest closed ones, and only on the order of 7% of Crossref works carry a deposited abstract. The open channel that should have absorbed the loss was always partial, and the scraped channel that masked that partiality is now closing.

Even the abstracts that remain are partly corrupt. Kim, Holst and Ginis (2026) assessed 10,000 sampled OpenAlex abstracts and found **12% of records flagged `has_abstract=True` do not contain a valid scientific abstract** (PubMed conclusion-only snippets, repository metadata, web-scrape chrome, truncation, placeholders, wrong genre). The rate falls over time but the failures cluster by era, source and document type, so they bias rather than merely add noise.

The same failure is measurable inside this corpus. Scopus exports the literal string `[No abstract available]` where it holds no abstract, and the ladder that picks each record's working abstract from the three channels originally took the first channel whose abstract field was non-empty. Twenty-three characters of nothing are non-empty, so the ladder stopped at Scopus, wrote the placeholder into the working field, and never looked at the OpenAlex or Web of Science text sitting in the same row. 1,417 records were counted as carrying an abstract on that basis, 1,382 of them holding a placeholder of that family and 35 a fragment shorter than 100 characters; 1,393 of the 1,417 fall inside the 2000–2025 analysis window. The ladder now falls through on usability rather than on non-emptiness, and re-running it recovered real text for 664 of those records from a channel it had been skipping, leaving the rest as the absences they always were. Reported in-window coverage fell from 97.4% to 96.3%, and the residual of works with no abstract anywhere rose from 1,859 to 2,595. Neither movement is a loss of data. Coverage measured on text a classifier can actually read moved the other way, from 95.4% to 96.3%, and the higher figure it replaces was an artefact of counting placeholders as abstracts. A later sweep of the open abstract APIs at Semantic Scholar, Crossref and Europe PMC supplied replacements for a further 2,766 records

whose held abstract failed the pipeline's own quality checks, which test for truncation, page furniture, body text, a foreign-language string and an abstract belonging to a different publication. This corpus's instance of what Kim, Holst and Ginis measure is therefore both larger than a placeholder count alone would show and, for most of the affected records, repairable. Where coverage stands after all of that is reported in Notebook 3..

For a baseline, Culbert et al. (2025) found OpenAlex abstract coverage of about 87% against over 92% for Web of Science and Scopus. The gap to the curated databases was already real before the latest withdrawals; it is widening month by month, and specifically for the closed-access, large-publisher, recent content the project cares about. **That is the case for using all three sources rather than OpenAlex alone:** the curated databases hold abstracts and curated keyword fields that OpenAlex is losing, while OpenAlex holds the open citation graph and the per-work topic net the curated databases lack. Each covers the others' blind spots.

1.4 The selection design

Because no single query is both precise and complete, the project operationalises "the trust we want" at **three moments**, each with different strengths and blind spots, and combines them:

1. **Query-time selection in the curated databases** – a deliberately precise keyword design run on Scopus and Web of Science.
2. **Query-time selection in OpenAlex** – selection by the work's machine-assigned *topic* rather than by words, over a curated union of trust-bearing topics.
3. **Record-level classification after retrieval** – a transparent rule that reads each retrieved record and labels the *kind* of trust it is about.

The first two decide what enters the corpus and are set out here; the third – the trust-sense classifier – is the subject of Notebook 2.. All queries are archived verbatim in `data/queries/`.

1.4.1 Selection in the curated databases: a precise keyword design

Scopus and Web of Science are searched with the same logic in each database's syntax: a block of explicit **sense phrases** ("social trust", "political trust", "institutional trust", …), a **proximity block** pairing the trust word family with the objects of social-scientific trust (government, institutions, police, courts, media, science, experts, strangers, people, citizens, …) within three words, and a guarded **confidence-in-institutions** clause that picks up the survey-research idiom ("confidence in parliament") without admitting the everyday sense of confidence. "Social capital" is deliberately excluded here, because it re-enters through the OpenAlex topic union and the reference miner; "trustee"/"trusteeship" are kept out by enumerating the trust terms rather than truncating `trust*`. There is no discipline restriction – breadth is cross-disciplinary, and precision is imposed later by the classifier and the screen. The range is 1999–2026 (buffer years around the 2000–2025 analysis window), English, articles and book chapters.

Scopus (Advanced Search):

```
TITLE-ABS-KEY(
  ( {social trust} OR {political trust} OR {institutional trust} OR {interpersonal trust}
      OR {generalized trust} OR {generalised trust} OR {particularized trust} OR
{particularised trust}
    OR {dispositional trust} OR {system trust} OR {civic trust} OR {personal trust}
    OR {general trust} OR {public trust}
      OR {political distrust} OR {institutional distrust} OR {social distrust} OR
{public mistrust} )
  OR ( (trust OR trusted OR trusting OR trusts OR trustworth* OR mistrust* OR distrust*)
       W/3 (government* OR governance OR parliament* OR politician* OR institution* OR
police OR court*
              OR judiciary OR authorit* OR media OR science OR scientist* OR expert*
OR stranger*
               OR others OR neighbour* OR neighbor* OR people OR citizen* OR society
OR state) )
  OR ( confidence W/3 (government* OR parliament* OR institution* OR police OR court*
OR politician* OR authorit*) )
)
```

```
AND PUBYEAR > 1998 AND PUBYEAR < 2027 AND LANGUAGE(English) AND (DOCTYPE(ar) OR
DOCTYPE(ch))
```

Web of Science (Core Collection; `TS=` reaches title, abstract, author keywords and the cited-reference-derived Keywords Plus; `?` is a single-character wildcard):

```
TS=(
  ( "social trust" OR "political trust" OR "institutional trust" OR "interpersonal trust"
      OR "generali?ed trust" OR "particulari?ed trust" OR "dispositional trust" OR
"system trust"
    OR "civic trust" OR "personal trust" OR "general trust" OR "public trust"
      OR "political distrust" OR "institutional distrust" OR "social distrust" OR
"public mistrust" )
  OR ( (trust OR trusted OR trusting OR trusts OR trustworth* OR mistrust* OR distrust*)
       NEAR/3 (government* OR governance OR parliament* OR politician* OR institution*
OR police OR court*
               OR judiciary OR authorit* OR media OR science OR scientist* OR expert*
OR stranger*
                OR others OR neighbour* OR neighbor* OR people OR citizen* OR society
OR state) )
   OR ( confidence NEAR/3 (government* OR parliament* OR institution* OR police OR
court* OR politician* OR authorit*) )
)
AND PY=(1999-2026) AND LA=(English) AND DT=(Article OR "Book Chapter")
```

1.4.2 Selection in OpenAlex: by topic and broad keyword search across titles and abstracts

OpenAlex is queried differently, because its strength is the machine-assigned **topic** of each work (section 1.2). Rather than restrict to one topic – which would capture only the trust-*defining* core – the pull uses a curated **union of twelve topics** where trust research concentrates across domains, intersected with the trust keyword family so that off-target members of those topics are filtered at the source. The twelve topics:

Table 1.1: The twelve OpenAlex topics in the discovery union.

Topic ID	What it covers
T10646	Behavioural-economics trust games
T10827	Patient–provider communication
T10833	Vaccine hesitancy
T10953	E-government
T11076	Policing and legitimacy
T11239	Social capital
T11541	Oxytocin / neuro-trust
T11573	Risk perception
T11696	Conflict and negotiation
T12028	Knowledge management
T12121	Cooperative studies
T12312	Culture, economy and development

The live API call (1999–2026, English, articles and book chapters):

```
GET https://api.openalex.org/works?filter=
     primary_topic.id:T10646|T10827|T10833|T10953|T11076|T11239|T11541|T11573|T11696|
T12028|T12121|T12312,
    title_and_abstract.search:trust OR distrust OR mistrust OR trustworthy OR
trustworthiness OR distrustful OR mistrustful OR untrustworthy,
  language:en,
  type:article|book-chapter,
  publication_year:1999-2026
```

The keyword search runs on *titles* and *abstracts* (and it seems to pick up keywords present in an abstract even when the abstract is now protected and not included in the pulled metadata). A by-DOI **crosswalk** then fetches the full OpenAlex record for every curated (Scopus/WoS) work not already in the topic pull, so the citation graph and OpenAlex metadata attach to every work. The collection scripts (`Scripts/collect/openalex-topic.R`, `Scripts/collect/openalex-crosswalk.R`) hold the exact parameters.

1.5 Completing the only-OpenAlex region: the curated DOI-backfill

Selection by these three channels leaves one large, awkward group: works the OpenAlex topic search returned but the curated keyword searches did not – the **only-OpenAlex** (pull-only) region. Two terms keep the next step straight. The **OpenAlex pull** is what the topic search returned; the **OpenAlex database** is what OpenAlex holds regardless of any query; symmetrically, the **curated searches** are what the Scopus/WoS keyword queries returned, and the **curated databases** are those indexes in full. A pull-only work was not *returned* by a curated search, but its DOI may nonetheless *exist* in a curated database – the search simply did not match it.

A **reverse check** establishes which: for every pull-only work with a DOI, the live Scopus and Web of Science APIs are asked whether that DOI exists in their database at all (`Scripts/collect/reverse-check.py`). The works whose DOI does exist there – present in a curated database but missed by the curated search – are **recoverable**: their abstract and curated keyword fields are sitting in a record the keyword search never returned. A bulk by-DOI pull then fetches those full curated records (`Scripts/collect/backfill-build-queries.py` builds the queries; the exports are re-merged in assembly), enriching the pull-only works with curated abstracts, author/index keywords and Keywords Plus, and document types they otherwise lacked.

Two safeguards matter. The backfill is **metadata only**: it fills the `Scopus_*` / `WoS_*` fields and is flagged (`scopus_backfilled` / `wos_backfilled` / `curated_backfill`), but it never sets the retrieval-sense membership flags `in_Scopus` / `in_WoS` – those continue to mean "the *search* returned it", so every coverage, overlap and added-value result in Notebook 3. is unaffected by the backfill. And it respects the licence boundary: recovered abstracts are tagged licensed and used for local classification, never redistributed and never sent to an external service. What this backfill recovers, and how much of the abstract gap it closes, is reported in Notebook 3..

1.6 The per-article licence warrant

The second of those safeguards states one half of a rule. The other half is what permits any text to leave this machine at all, and it cannot be read off the presence of the text. An abstract sitting in OpenAlex is evidence that OpenAlex holds the string. It is not evidence that anyone has granted a right to copy that string, and the two are easy to conflate because an open aggregator feels like an open licence. The distinction matters here for the reason section 1.3 already gave: a large share of the abstracts OpenAlex holds are inheritances from Microsoft Academic Graph, deposited with no licence at all and attached to works that are otherwise closed, which is precisely the class publishers are now withdrawing. Of the 52,196 works in the analysis window that hold a usable OpenAlex abstract, 23,910 – 45.8% – sit on works OpenAlex itself records as closed access. A rule that read presence as permission would send the text of nearly twenty-four thousand closed-access works to an external service.

The warrant is therefore fetched per work rather than inferred from access status. `Scripts/collect/openalex-licences.py` asks OpenAlex, for each of the 70,940 corpus works carrying an OpenAlex identifier, what licence is recorded on the work's primary location, falling back to its best open-access location where the primary carries none. The pass ran on 30 July 2026, and nothing but work identifiers left the machine. 25,808 of those works carry an actual licence string; the remaining 45,132 carry none,

which is what a closed work looks like from outside. The result is the column `OA_licence`, folded into the corpus at assembly and stamped with the date it was collected, since a publisher can retract or replace a licence and a warrant is only as current as the fetch behind it.

`abstract_externally_sendable` is the gate built on that column, and it is true only where both conditions hold: a usable OpenAlex abstract string is held for the work, and `OA_licence` names an explicit open licence. In the analysis window it is true for 21,527 of the 71,108 works. `other-oa` does not qualify. OpenAlex uses that value for a work that is open by some route with no recognised licence attached, which is evidentially the same position as no licence at all, so its 1,421 window works are treated as unwarranted. `abstract_licence`, the column that tags a backfilled abstract as licensed, names the channel that supplied a record's working abstract and carries no warrant of its own.

The warrant sorts into three tiers, each admitting everything the tier above it admits and adding one further restriction, so that tightening or loosening the boundary later is a filter on one column rather than a redraw of the population:

Table 1.2: The three nested warrant tiers in the analysis window.

Tier	Licences it adds	Added	Cumulative
strict	cc-by, cc-by-sa, cc0, public-domain	15,393	15,393
non-commercial	cc-by-nc, cc-by-nc-sa	2,824	18,217
no-derivatives	cc-by-nd, cc-by-nc-nd	3,276	21,493

The full tier, 21,493 records, is the set whose title and abstract may be sent to an external service for classification. It falls a little short of the gate's 21,527 because these counts apply the higher of two floors. An abstract needs 100 characters to count as usable at all, and 200 to carry a judgement about a study's methodology. The no-derivatives tier is the one worth a second look before anything is published from it, because a coding label derived from an abstract is arguably not a derivative work of that abstract. That is a question about what may be redistributed rather than about what may be read, and keeping the tiers in a single column leaves it answerable later without redrawing anything.

Read alongside the backfill safeguard, the two halves of the rule are symmetrical. What the gate opens is the OpenAlex text of a work whose own licence permits it, and only that. The curated abstracts recovered by the backfill arrive under the Scopus and Web of Science agreements and are never what is sent, whatever licence the work itself carries. Works with no warrant are not dropped. Their abstracts are read locally instead, by an open-weight model on university hardware. Eligibility is decided per article, in one column, as of a stated date. How the analysis window divides between the two channels is reported in Notebook 3..

2. Defining and detecting ‘trust’

ℹ Status: conceptual (scripted and reproducible)

The record-level classifier is `Scripts/trust_context.py`; it is applied in the derive stage (`Scripts/derive/variables.py`) and documented column-by-column in the derivation registry (`Scripts/derivations.yaml`). This notebook describes the classifier on its own terms; what it finds across the assembled corpus is reported in Notebook 3..

2.1 Selection decides what enters; the classifier decides what each record is about

Building a dataset of social-trust research runs into a difficulty the databases notebook (Notebook 1.) sets out and our own experiments kept confirming: **no single way of selecting records is both precise and complete.** A broad subject filter is high-recall but full of off-target trust (consumer trust in brands, user trust in devices, trust in network protocols); a single well-chosen topic is clean but captures only a fraction of the trust literature, because trust is studied *inside* hundreds of substantive topics rather than in one; and a keyword search is precise but blind to work that never says “trust” in the fields the search can reach.

The project’s answer, set out in the databases notebook, is to stop looking for that one query and instead operationalise “the trust we want” at **three moments**: a precise keyword design run on Scopus and Web of Science, a topic-based pull from OpenAlex, and a record-level classification *after* retrieval. The first two – the selection design that decides what enters the corpus – are described there (section 1.4). This notebook is the **third moment**: the transparent rule that reads each retrieved record and labels the *kind* of trust it is about. It runs over every record the collection returns, and what it writes travels with that record into the stages that follow, so it is set out here conceptually; the distribution it produces over the assembled corpus is reported in the corpus chapter (Notebook 3.), and the screening stage that reads its output is described in Notebook 6..

2.2 The trust-sense classifier

The classifier decides what each record *is about*. It is a transparent, rule-based tool (`Scripts/trust_context.py`), not a model: for each record it scans the title, abstract and the merged keyword fields for the trust word family, extracts the **qualifier before** a trust word (“social trust”) and the **object after** trust/confidence/faith/reliance + in/on/of (“trust in government”), and maps each to a fine **`trust_sense`** drawn from a lexicon grounded in the survey-research literature (the WVS/ESS confidence-in-institutions battery and the standard distinctions: generalised/social, interpersonal, institutional/systems, political, legal/justice, science/experts, media/information, religion, civic/community, and the health-domain clinical/relational sense; against the off-construct senses: consumer/marketing, technology, security/computing, commercial-finance).

A coarse **`trust_scope`** is then rolled up from that sense, and it is the label the later stages read:

- **`in-scope`** – the dominant matched sense is one of the social-science trust senses (health-domain trust is in-scope).
- **`out-of-scope`** – the dominant sense is out of the construct (consumer, technology, security, finance).
- **`undetermined`** – no qualifier or object matched, so the rule did not pin a sense.

Three properties of this scheme matter for how it is used:

1. **The three states are asymmetric.** `in-scope` and `out-of-scope` mean *the rule reached a decision*; `undetermined` means *the rule was silent* – it does **not** mean “no trust” or “irrelevant”. It is the large

residual precisely because trust is often discussed without a lexical cue the rules catch, and because some records have only a title to read.
2. **`out-of-scope` is a lower bound.** An off-construct study whose object is not matched falls into `undetermined`, not `out-of-scope`, so the out-of-scope count understates the true off-construct share. `in-scope`, by contrast, is high-precision: the lexicon is specific.
3. **It is a prior, not a verdict.** The rule is cheap and auditable, and it labels every record in the corpus, but no record is admitted or dropped on its say-so. The three states are carried forward as a feature: the screening coder reads them alongside the record itself, and the filters further downstream consult them. Where a coder who has read the paper disagrees with the lexicon, the coder's judgement governs.

2.2.1 Reading the keyword fields, and recording where the sense came from

Because a record can be silent on trust in its title and abstract yet clearly trust-tagged in its index terms – Web of Science's Keywords Plus, derived from cited references, is a recurring example (Notebook 1.) – the classifier reads the **full merged keyword set** (author keywords, Scopus index keywords, WoS Keywords Plus, OpenAlex keywords), not just title and abstract. To keep that transparent, it classifies the title-and-abstract text and the keyword text **separately** and records the provenance: `trust_sense_text`, `trust_sense_keywords`, a `sense_basis` flag (text / keywords / both / none), and `scope_conflict`, which is set only when the two signals disagree on scope itself (one in-scope, the other out-of-scope). On a genuine scope conflict the **text wins** the assignment – it is the authors' own framing – and the flag marks the case for review; small sense-only differences within the same scope are not flagged, so they raise no spurious conflicts. How much the keyword fields add, and how the triage distributes across the assembled corpus, is reported in Notebook 3..

2.2.2 A tested exclusion: OpenAlex `concepts`

One candidate input was considered and rejected on evidence rather than assertion. OpenAlex assigns each work a list of machine-generated **`concepts`**, which are broad and noisy. Feeding them to the classifier would resolve only about a tenth of a percent of the undetermined works, and it would do so *harmfully*: the works it "resolves" include computing and security studies pulled wrongly in-scope through concepts like "Trust management (information system)" – for instance a blockchain trust-management scheme and an IoT trust-based routing scheme, both of which are exactly the off-construct technical trust the triage should exclude. The negligible gain and the false-positive cost are why the classifier uses the curated keyword fields but not `concepts`. (The experiment is `Scripts/experiments/concepts.py`.)

2.3 The full classifier criteria

So the rules can be examined and reasoned about without reading the code, the complete criteria set is reproduced here (generated from `Scripts/trust_context.py`; the lexicon currently holds 15 senses – 11 in-scope, 4 off-target – and 49 multiword objects).

How a record is read. Two regular expressions scan the lower-cased title + abstract + merged keywords. The first captures the **qualifier immediately before** a trust word ("social trust" yields `social`); the second captures the **object after** a trust word ("trust/confidence/faith/reliance in/on/of X", up to three words – "trust in the government" yields `government`). Each captured qualifier or object is mapped to a fine **sense** – multiword objects are tested first, then single tokens against the lexicon below, with stopwords skipped and object-only terms counted only as objects. The most frequent matched sense becomes `trust_sense`; `trust_scope` is **in-scope** if that sense is one of the eleven social-science senses, **out-of-scope** if one of the four off-target senses, and **undetermined** if nothing matched.

2.3.1 In-scope senses (`trust_scope = in-scope`)

- **generalised/social** – generalized, generalised, social, diffuse, general, thin, particularized, particularised, horizontal
- **interpersonal** – interpersonal, mutual, dyadic, relational
- **in-people/others** – people, others, strangers, stranger, humanity, neighbours, neighbors, humans, everyone
- **institutional/systems** – institutional, institutions, institution, organizational, organisational, bureaucratic, bureaucracy, system, systems, authorities, authority, regulators, regulatory, agencies, agency,

unions, union, ngos, ngo, companies, corporations, corporate, business, banks, bank, abstract, expert, nhs
- **political** – political, government, governmental, politicians, politician, democracy, democratic, state, parliament, parliamentary, regime, electoral, public, council, congress, senate, citizen, parties, party, politics, governance
- **legal/justice** – police, judicial, judiciary, court, courts, legal, justice, law, lawyers
- **science/experts** – science, scientists, scientific, experts, expertise, academia, academic, universities, university, research, researchers, knowledge
- **media/information** – media, news, press, journalists, journalism, information, misinformation, television, newspapers
- **religion** – church, churches, religion, religious, clergy, god, faith, mosque, temple, priests, spiritual
- **civic/community** – civic, community, communal, neighbour, neighbor, neighbourhood, neighborhood, associations, voluntary
- **clinical/relational-health** – patient, patients, physician, physicians, doctor, doctors, nurse, nurses, clinician, clinical, provider, providers, dentist, therapist, medical, health

2.3.2 Off-target senses (`trust_scope = out-of-scope`)

- **consumer/marketing** – consumer, consumers, customer, customers, brand, seller, sellers, vendor, vendors, purchase, shopping, retail, ecommerce, advertising, marketing, buyer, loyalty, marketplace, marketplaces, influencer, influencers
- **technology** – technology, technological, automation, robot, robots, algorithm, algorithms, ai, artificial, autonomous, website, websites, app, apps, chatbot, platform, automated, machine, machines, internet, device, devices, smartphone, gpt, chatgpt, llm, llms, generative, metaverse, driverless, drone, drones, wearable, wearables, recommender
- **security/computing** – computing, access, cyber, blockchain, cryptographic, protocol, node, nodes, iot, cloud, sensor, sensors, software, wireless, authentication, encryption, phishing, malware, biometric, biometrics
- **commercial-finance** – financial, investor, investors, fintech, credit, insurance, crypto, cryptocurrency, blockchain, market, trading, bitcoin, stablecoin, defi, derivatives

2.3.3 Multiword objects (matched as substrings of the captured object, before single tokens)

- **institutional/systems** – health system, healthcare system, health care system, public health, health authorities, education system, welfare state, welfare system, civil service, armed forces, public institutions, public sector, public administration, european union, united nations, world bank, international organizations, banking system, financial system, financial institutions
- **legal/justice** – criminal justice, justice system, legal system, rule of law
- **political** – political parties, political system, local government, central government, national government
- **science/experts** – scientific community, higher education
- **media/information** – social media, mass media, news media
- **technology** – machine learning, deep learning, neural network, large language model, language model, generative ai, self driving, autonomous vehicle, autonomous vehicles, recommender system, recommendation system, virtual assistant
- **security/computing** – smart contract
- **consumer/marketing** – social commerce, e commerce

The off-target multiwords are deliberately multiword-only where a single token would collide with an in-scope sense: “deep learning” and “neural network” are off-target, but bare `deep` (“deep trust”) and `neural` are not in the lexicon, and `recommendation` (“expert recommendation”) is reached only through “recommender system” / “recommendation system”.

2.3.4 Special handling

- **object-only** (counted only as the object of “trust in X”, never as a qualifier, so “an NHS Trust” – an organisation – is not read as institutional trust): `nhs`.

- **stopwords** (skipped as qualifiers and objects): a, an, and, as, based, between, both, build, building, built, each, for, her, high, his, how, in, is, its, lack, less, level, loss, low, more, of, on, other, our, s, such, than, that, the, their, this, to, very, what, with.

The core algorithm (`Scripts/trust_context.py`)

```
# qualifier directly before a trust word ("social trust" -> "social")
MODIFIER = re.compile(r"\b([a-z]+)\s+(?:dis|mis|un)?trust(?:worthy|worthiness|ed|
ing|s)?\b", re.I)
# object after trust/confidence/faith/reliance/belief in|on|towards|of  (up to
3 words)
OBJ    =    re.compile(r"\b(?:(?:dis|mis|un)?trust(?:ed|ing|s)?|confidence|faith|
reliance|belief)"
                      r"\s+(?:in|on|towards?|of)\s+(?:the\s+|their\s+|our\s+|a\s+|
an\s+|its\s+)?"
                  r"([a-z]+(?:\s+[a-z]+){0,2})", re.I)

def sense_of(phrase):
    for mw, s in MULTIWORD.items():        # multiword objects win
        if mw in phrase:
            return s
    for tok in phrase.split():            # else first single token in the lexicon
        if tok in KW2SENSE:
            return KW2SENSE[tok]
    return None

def classify(text):
    t = str(text).lower()
    mods = [m.group(1).strip() for m in MODIFIER.finditer(t)]
    objs = [m.group(1).strip() for m in OBJ.finditer(t)]
    hits, kept = Counter(), []
    for p in mods:                        # qualifier before trust ("social trust")
        if p in STOP or p in OBJECT_ONLY:
            continue
        s = sense_of(p)
        if s: hits[s] += 1; kept.append(p)
    for p in objs:                         # object after "trust/confidence in X"
        if p in STOP:
            continue
        s = sense_of(p)
        if s: hits[s] += 1; kept.append(p)
    return (hits.most_common(1)[0][0] if hits else "unspecified"), kept
```

2.4 Limitations and what builds on this

The classifier is deliberately simple, and its limits are the reasons it is a *first* layer rather than the last word: it depends on the abstract (a title-only record has little to read), it takes the most frequent matched sense rather than weighing them, its lexicon is finite, and `out-of-scope` is a lower bound. None of this is a defect for its purpose – a transparent, reproducible triage and feature – but it is why the design treats the rule’s output as a prior. Its dependence on the abstract is also a dependence on the abstract’s quality, which is why the triage is re-derived from the corpus every time the pipeline runs rather than stored once and inherited. When the August 2026 retrieval sweep replaced or supplied 2,843 abstracts (Notebook 3.), 49 of those records changed `trust_scope`, the movement running towards `undetermined` as the rule found itself reading a whole abstract in place of a fragment.

2.4.1 What the triage feeds

The rule labels all 75,302 records in the corpus, 71,108 of them inside the 2000–2025 analysis window, and three things downstream read what it writes.

The **calibration sample** on which the screening codebook was drafted and tested – 1,500 records set aside in advance in blocks of 250, three of which were coded and re-coded as the rules were revised – was drawn stratified on `trust_scope` crossed with document type and decade. The rules were therefore written against records the lexicon had left undetermined and records it had read as off-construct, rather than against the clean in-scope ones alone.

The **coder** reads `trust_snippet`, the context window around each trust mention, which the same module produces (`trust_snippets()` in `Scripts/trust_context.py`) and which sits in the coding record beside the title, the abstract and the four keyword fields, kept separate there by provenance where the classifier reads them merged. This is the part of the classifier a human uses directly. It puts every trust mention in front of the reader in its own sentence, whether or not the rule managed to resolve a sense from it.

The **filter into the methodological-extraction pool** consults `trust_scope` itself. A record the rule places out-of-scope is dropped, unless the coder judged it central to social trust, in which case it stays and keeps its out-of-scope label in the data so the disagreement remains findable. Every `undetermined` record is kept, on the reasoning that a rule which failed to resolve a sense has said nothing about the record.

2.4.2 The screening codebook

The screening decision itself is made under a separate instrument: a human codebook, frozen at version 3, carrying two variables that a coder judges from the record. **`h_class`** asks whether the work is about social trust and how centrally; **`h_methodology`** asks what kind of study it is. Neither is a property of the record's wording. The first depends on what work the trust construct does in the paper's own claims – whether the claim sentences single it out or report it level with its neighbours – while the second depends on what design the evidence rests on. A rule that counts qualifiers and objects can pose neither question. The coded variables are held in their own columns and never overwrite `trust_sense` or `trust_scope`, so the rule's reading and the coder's stay separately auditable and their disagreements can be measured instead of being absorbed.

Which coding track a record enters is decided by the licence its publisher attached to it rather than by anything the triage said. Records whose abstracts carry an explicit open licence are coded by an external service; records holding a usable abstract without that warrant go to an open-weight model on the university's own hardware; and the records with no abstract at any index are coded from title and metadata alone. The codebook, the calibration waves that fixed it, the reliability the coding achieves and the three tracks are the subject of Notebook 6.. What belongs here is the division of labour: the rule labels every record cheaply and auditably, and the coder's effort goes on the two judgements it cannot make.

2.5 Reproducibility

The classifier is `Scripts/trust_context.py`, applied by `Scripts/derive/variables.py` and documented column-by-column in `Scripts/derivations.yaml` (rendered in the codebook, Notebook 5.). The selection queries that decide what enters the corpus are set out in the databases notebook (section 1.4) and archived verbatim in `data/queries/`. The concepts experiment is `Scripts/experiments/concepts.py`. The distribution the classifier produces over the corpus is reported and tabulated in Notebook 3., and the screening codebook and coding runs that consume these columns are documented in Notebook 6..

3. The THEME corpus

i Status: scripted and reproducible

The corpus is built by the pipeline in `Scripts/` (collection → `collect/openalex-licences.py` → `collect/retrieve-abstracts.py` → `assemble/1-spine.py` → `assemble/2-abstracts.py` → `derive/variables.py`), which writes the analysis dataset `data/final/corpus_derived.parquet`. Every table and figure in this notebook is computed from that `corpus_derived` dataset, imported via the R `arrow` package.

3.1 What the 2015 audit established

Before this corpus was assembled, a focused audit on a single complete year (2015) tested the design across the three databases. It is not re-run here – it is preserved, runnable, in `Scripts/2015-audit/` – but it settled four things that motivated the collection approach, all of which the full corpus below then confirms at scale:

1. **Coverage is near-total.** OpenAlex contained essentially all (98.9–100%) of the trust research the curated Scopus and Web of Science searches returned, so OpenAlex can serve as the discovery substrate and citation backbone.
2. **No single query retrieves it all.** The OpenAlex topic-union retrieved under a tenth of the curated trust OpenAlex itself contains, and Web of Science was largely a subset of Scopus – because trust is studied *inside* hundreds of substantive topics, not in one.
3. **The databases are complementary.** Each channel held in-scope works the others missed, so recall has to come from merging all three by DOI, with precision imposed afterwards by the classifier and the screen.
4. **The only-OpenAlex works are mostly recoverable.** About two-thirds of the works only OpenAlex returned existed in a curated database after all – missed by the keyword *query*, not absent from the *database* – which is what later makes the curated by-DOI backfill (section 1.5) worthwhile.

This notebook applies the same three searches (Notebook 2., section 1.4) to the whole period and reports the result not as a test but as the **assembled corpus** the rest of the project works from. To resolve publication-year boundary mismatches across databases, the pull spans 1999–2026; 1999 and 2026 are buffer years, and the analysis window is **2000–2025** (a work enters the window if *any* source dates it inside it). Of 75,302 unique works in the full pull, **71,108 fall in the analysis window**; the rest are dated only to a buffer year in every source.

Two terms recur throughout, with the same meaning as in the databases notebook (section 1.5). The **OpenAlex pull** is the set of works the twelve-topic search returned; the **OpenAlex database** is what OpenAlex holds regardless of any query. The same split applies to the curated side – the **curated searches** (the Scopus and Web of Science keyword queries) versus the **curated databases** (those two indexes in full). 'Recoverable' therefore means present in a curated database but not returned by the curated searches, and a work can sit in the OpenAlex-pull-only region yet still exist in a curated database – a distinction that matters directly for the abstract gap (section 3.10).

3.2 The collection

Each search is deduplicated within its own source before any comparison.

Table 3.1: Works returned by each search, 2000–2025 analysis window (deduplicated within each source).

Database	Records
Scopus	48,626
Web of Science	35,444
OpenAlex pull (topic-union)	23,807

Scopus is the broadest keyword search, Web of Science the more selective, and the OpenAlex twelve-topic union – which captures the trust-*defining* core rather than trust-as-studied-everywhere – the smallest.

3.3 Coverage: does the OpenAlex database contain the curated records?

For every Scopus and Web of Science record with a DOI, the by-DOI crosswalk asks whether that DOI exists anywhere in the OpenAlex database – a query-independent check, read offline from the topic pull plus the crosswalk.

Table 3.2: Presence of each curated record in the OpenAlex database, full corpus.

Source	DOI'd records	In OpenAlex	In OpenAlex %	Genuinely absent	Absent %	No-DOI
Scopus	45,898	45,179	98.4	719	1.6	2,728
Web of Science	34,326	34,007	99.1	319	0.9	1,118

The single-year result holds across the quarter-century: genuine absence is **1.6%** of DOI'd Scopus records and **0.9%** of Web of Science (an upper bound – a few malformed DOIs fail to resolve). **The OpenAlex database contains essentially all the trust research the curated searches return**, so it can serve as the discovery substrate and citation backbone, with curated records recoverable by DOI.

3.4 Overlap and uniqueness

Coverage is near-total, but the *retrieved* sets overlap modestly.

Table 3.3: Pairwise retrieval overlap across the window corpus.

Comparison	Overlap
Scopus found in OpenAlex	5,480 (11.3%)
WoS found in OpenAlex	4,687 (13.2%)
WoS found in Scopus	30,843 (87%)
Scopus found in WoS	30,843 (63.4%)

The OpenAlex pull retrieves about 11–13% of the curated trust the database overwhelmingly contains, and Web of Science is largely a subset of Scopus (87%) – both curated searches run the same precise query, while the OpenAlex pull draws a different boundary. *Why* each search returns less than its database holds was settled at the single year (section 3.1) and is consistent with the corpus: the dominant filter is substantive – trust is scattered across topics outside the twelve-topic net – with document-type exclusions accounting for much of the reverse gap.

3.5 The Venn regions: does OpenAlex's classifier add value?

Each work sits in one of seven Venn regions and is triaged by the trust-sense classifier (Notebook 2.) into in-scope, out-of-scope or undetermined.

Table 3.4: The window corpus by Venn region and trust_scope.

region	n	in-scope	out-of-scope	undetermined
only OpenAlex	17,881	5,055	202	12,624
only Scopus	16,544	8,754	1,047	6,743
only WoS	4,155	2,572	126	1,457
OpenAlex+Scopus	1,239	917	20	302
OpenAlex+WoS	446	335	4	107
Scopus+WoS	26,602	16,781	733	9,088
all three	4,241	3,496	31	714
Total	**71,108**	**37,910**	**2,163**	**31,035**

The added-value finding replicates with force. OpenAlex's topic search found **17,881 trust works that both curated keyword searches missed**; just 1.1% are out-of-scope, 28% are in-scope, and the rest await the content screen. The conclusion is bidirectional – the only-Scopus region holds 8,754 in-scope works the other two searches miss, the only-WoS region 2,572 – so no single channel is sufficient; the curated keyword searches contribute the largest blocks of uniquely-relevant recall, and works found in all three are the most clearly in-scope (82%).

3.6 The only-OpenAlex region: recoverable or genuinely unique?

The Venn table shows OpenAlex's topic search surfaced 17,881 works the curated searches missed, but not whether those works are *genuinely* outside the subscription databases or merely missed by the curated *queries*. A reverse check settles it, exactly as at the single year (section 3.1): for every only-OpenAlex work with a DOI, the live Scopus and Web of Science APIs were asked whether the DOI exists in their database at all (`Scripts/collect/reverse-check.py`), capturing the curated document type where it does.

Table 3.5: The only-OpenAlex region split by the reverse check against the curated databases.

Of the only-OpenAlex works	n
with a DOI, in Scopus	9,459
with a DOI, in Web of Science	7,239
with a DOI, in either curated database (recoverable)	9,946
with a DOI, in neither (genuinely OpenAlex-unique)	5,672
no DOI (unverifiable)	2,263

Table 3.6: Curated document type of the recoverable only-OpenAlex works (top genres).

curated document type	n
Article	7,288
Book Chapter	616
Conference Paper	505
Review	366
Editorial material	333
Meeting	290
Abstract	176
Article; Meeting	94

The single-year pattern holds at scale. Of the only-OpenAlex works with a DOI, **9,946 (64%) are recoverable** – they exist in Scopus or Web of Science, the curated *queries* simply did not return them – and only **5,672 (36%) are genuinely outside both subscription databases**, OpenAlex's true unique contribution. The recoverable works' curated document types are dominated by **Article (7,288) and Book Chapter (616)** – exactly the types the curated query accepts, so these are genuine in-scope articles the keyword or field match failed to return – with the genres the query excludes by document type (Conference Paper 505, Review 366, Editorial 333, Meeting 290) a substantial but minority share that OpenAlex admits under its coarse "article" type. These 9,946 recoverable works are the target of the curated by-DOI backfill (section 1.5), which reached 9,719 of them and enriched those with curated abstracts and keywords; what that recovers is reported in section 3.10.

3.7 Where the trust signal sits in each database

A question that bridges to whatever screens the corpus next: in which field of each database's record does the trust term actually appear? The classifier and any abstract-level screen can read a match in the title or abstract directly; a match only in the keyword fields – Web of Science's cited-reference-derived Keywords Plus above all (Notebook 1.) – is findable but needs the keyword fields read; and a work a search returned on a trust term now absent from its held title, abstract and keywords is the abstract-withdrawal tail made visible. The table scans each returned record's held fields for the trust word family (computed inline, not stored in the dataset; it approximates the finer categories of the single-year audit).

Table 3.7: Where the trust term sits in each source's held record (works that source returned).

Source	returned	title / abstract	keywords only	elsewhere / withheld
OpenAlex pull	23,807	21,432	184	2,191
Scopus	48,626	46,330	578	1,718
Web of Science	35,444	33,428	1,018	998

For most records the trust signal is in the title or abstract, where any screen can read it. The exceptions are instructive. **Web of Science carries the most keyword-only matches (1,018)** – its Keywords Plus, generated from the titles of a work's cited references, retrieves studies that engage the trust literature heavily without naming trust in their own text. **OpenAlex shows the largest "elsewhere/withheld" tail (2,191)**: works its topic search returned on a trust term that is no longer present in the held title, abstract or keywords – the abstract drought (Notebook 1.) made visible, since the search index remembered a match the record no longer exposes. Both are arguments for retaining the curated keyword fields and for the abstract backfill: the trust signal is not always where a naive title-and-abstract screen would look.

3.8 How trust research has grown, 2000–2025

The corpus shows what a single year cannot: the trajectory of the field and of each search's grip on it.

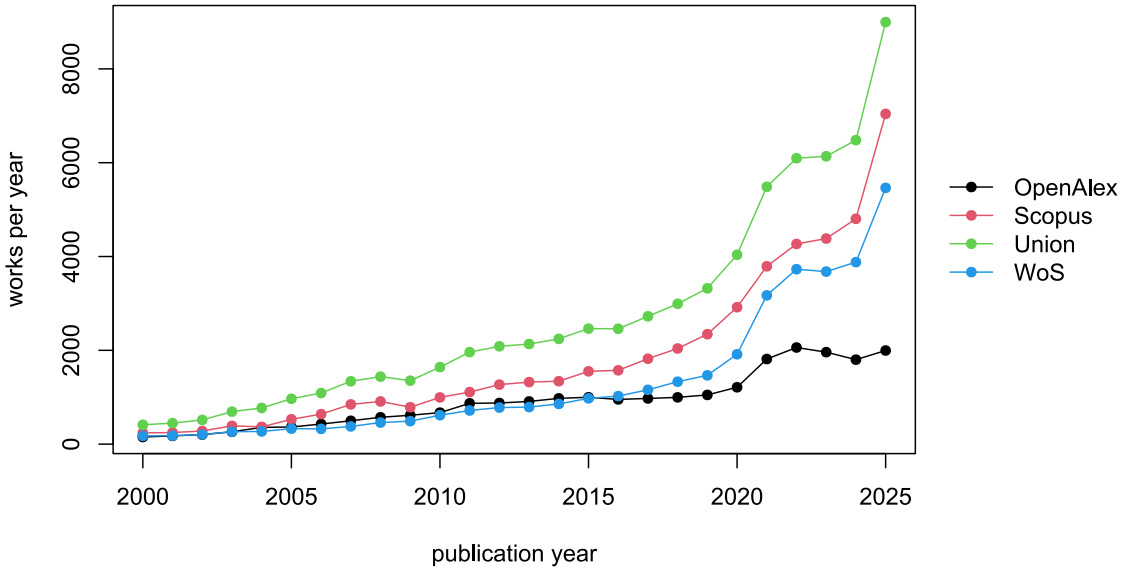

Figure 3.1: Trust-research works per year, 2000–2025: the deduplicated union and the share retrieved by each database search. The 2025 figure is inflated by online-first/in-press records and is read as a lower bound on the most recent year.

The same years, split by the trust-sense triage, show how the *relevant* share has held up as the field grew.

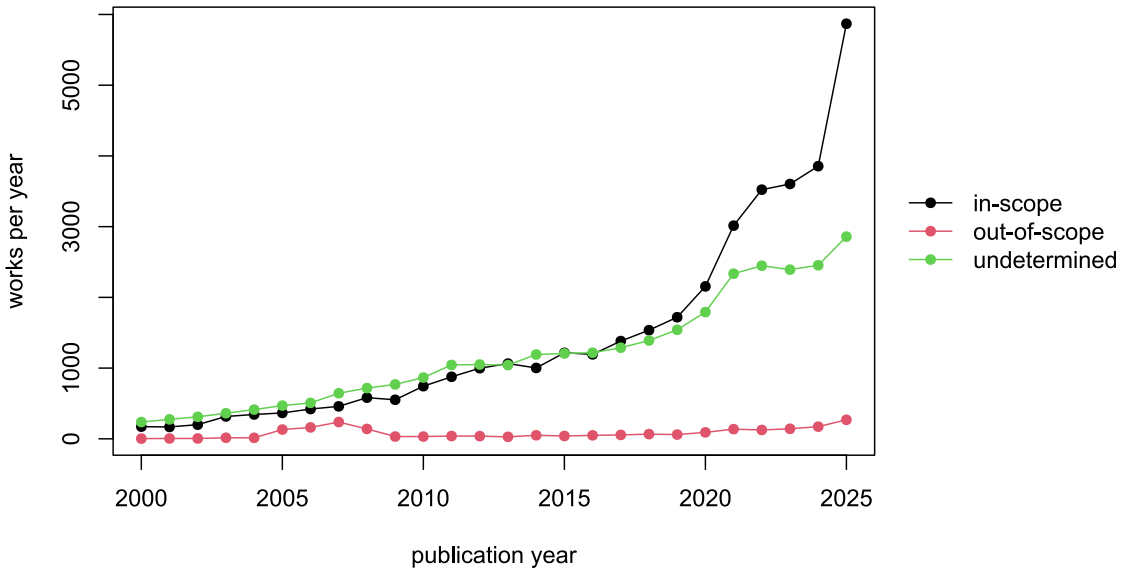

Figure 3.2: Trust-research works per year by trust_scope, 2000–2025: in-scope, undetermined and out-of-scope. The in-scope share holds steady across the period, so the growth is growth in relevant trust research, not in noise.

Three things stand out. The field has grown roughly twentyfold – from about 412 works in 2000 to several thousand a year by the early 2020s. The curated searches scale with that growth while the OpenAlex topic-union flattens (it levels off around 1,000–2,000 works a year and dips in the most recent years), consistent with a fixed twelve-topic net capturing a shrinking fraction of a widening field and with the abstract drought eroding the keyword clause OpenAlex depends on (Notebook 1.). The 2025 column is depressed for OpenAlex and inflated for the curated databases by online-first effects at the collection boundary, so the final year is read as provisional. And throughout, the **in-scope share holds steady** (Figure 3.2): the growth is growth in relevant trust research, not in noise, with out-of-scope work a thin, roughly constant band.

3.9 The trust-sense triage across the corpus

The record-level classifier (Notebook 2.) labels every work's `trust_scope` and records where the sense came from. Across the window corpus:

Table 3.8: The trust_scope triage over the window corpus, and where each resolved sense came from

trust_scope	n	%
in-scope	37,910	53.3
out-of-scope	2,163	3.0
undetermined	31,035	43.6
Total	**71,108**	**99.9**

Table 3.9: Where each work's combined sense came from (none = undetermined)

sense_basis	n
both	8,801
keywords	2,114
none	31,035
text	29,158
Total	**71,108**

The rule places **53.3%** of window works in-scope, **3.0%** out-of-scope, and leaves **43.6%** undetermined for the screen. Reading the keyword fields is not cosmetic: **2,114 works were classified from their keywords alone** (their title and abstract said nothing the rule could catch), and a further 8,801 had text and keywords agree; only **212 works** carry a scope conflict – a small, explicit review queue rather than a silent guess. The leading in-scope senses are political, institutional and generalised/social trust; the out-of-scope side is dominated by technology and consumer trust, exactly the contamination the triage is meant to hold back. Undetermined is the large residual by design (Notebook 2.) – it is the screen's main queue, not a verdict of irrelevance.

3.10 What is still without an abstract

The classifier and any downstream screen depend on the abstract, so the works that lack one are worth characterising. The residual reached its present size of **2,518** works in three passes, the middle of which raised it.

The curated by-DOI backfill (section 1.5) ran first. It recovered curated records for 9,719 window works, most of them in the recoverable only-OpenAlex block identified in section 3.6, and supplied an abstract wherever the curated record carried one. Measured on the abstract ladder as it then stood, the residual fell from 3,278 works to 1,859.

The ladder was then found to be accepting text it should not have. In 1,417 in-window records the field an abstract would occupy held a placeholder string or a fragment of under a hundred characters – a copyright notice, a running header, a truncated opening line – and the ladder had counted each of them as an abstract. Reclassifying them as the absences they always were took the residual up to 2,595 and measured coverage down from 97.4% to 96.3%. The fall records a correction: the text it removes was never usable. It is also this corpus's own instance of the failure Kim, Holst and Ginis report for 12% of the OpenAlex records flagged as carrying an abstract (section 1.3).

The third pass went outside the assembled sources. `Scripts/collect/retrieve-abstracts.py` queried 7,454 distinct DOIs against Semantic Scholar, Crossref and Europe PMC – identifiers only, with no held text leaving the machine – and folded 2,843 usable abstracts into the corpus as the active record. Seventy-seven of them filled records that held nothing at all, and the remaining 2,766 replaced texts that were present but failed the quality checks the ladder fix had introduced. Each retrieved abstract carries its source and its retrieval date in `abstract_retrieved`, and each is licensed for local processing only, so nothing folded in this way becomes externally sendable. The residual settled at **2,518** works and coverage at **96.5%**.

Table 3.10: Where each window work's working abstract comes from, after the curated backfill, the ladder fix and the open-API sweep.

abstract source	n
Scopus	53,806
OpenAlex	7,234
WoS	4,707
(none held)	2,518
SemanticScholar	2,212
Crossref	507
EuropePMC	124
Total	**71,108**

The curated exports carry the corpus: Scopus supplies 53,806 of the 68,590 working abstracts and Web of Science a further 4,707, against 7,234 from OpenAlex itself. That is the abstract drought (section 1.3) seen from the other side, and it shapes what the three passes leave behind:

Table 3.11: Characteristics of the works still with no abstract from any source (analysis window).

characteristic	value
document type (top 2)	article 1,488; book chapter 1,008
only in the OpenAlex pull (not returned by a curated search)	1,962
OpenAlex venue type (top 3)	journal 1,006; (none) 809; not in OpenAlex 282

The residual is **78% (1,962) pull-only**, so the works no curated search returned still dominate it, but the other 556 were returned by Scopus or Web of Science and have no abstract in those records either – 282 of them have no OpenAlex record at all, and 274 carry no DOI on which any retrieval could be attempted. Its largest single venue category is now `journal` (1,006 works, 981 of them typed as articles), ahead of the sourceless venues (809), ebook platforms (259) and book series (162) where book chapters sit. The residual therefore reaches into the mainstream of journal publishing: about a thousand of these are ordinary articles whose abstract is deposited nowhere the pipeline can reach, which is the abstract withdrawal described in section 1.3 working on the individual record. A further thousand are book chapters, where an abstract was often never written.

Splitting the pull-only majority by the reverse check (Table 3.5) shows what the three passes left behind:

Table 3.12: The residual abstract-less pull-only works split by the reverse check (after the backfill, the ladder fix and the open-API sweep).

Of the residual abstract-less pull-only works	n
with a DOI, in a curated database (no abstract there either)	367
with a DOI, in neither (genuinely OpenAlex-unique)	1,087
no DOI (unverifiable)	508
Total	**1,962**

The backfill reached most of the recoverable works and supplied an abstract wherever the curated record held one, leaving **367** whose curated record carried no abstract either. The larger part of what remains is the **genuinely OpenAlex-unique tail** (1,087, in neither curated database) and the **no-DOI tail** (508),

where no identifier exists on which to ask any of the six channels. All are retained and flagged rather than dropped. Retrieval has been exhausted for them – the open-API sweep queried every one that had a DOI to query – so they are screened from title and metadata alone, and the labels that come back are held as weak by design.

3.11 The assembled dataset

The pipeline writes the analysis dataset to `data/final/corpus_derived.parquet`, 75,302 rows by 177 columns; the assembled base before classification is `corpus_raw.parquet` in the same folder, the intermediate spine and the OpenAlex citation graph live in `data/intermediate/`, and every column is documented in `codebook.html` (Notebook 5., generated from the derivation registry). Parquet is the canonical format (read directly by these notebooks via R `arrow`); a CSV can be produced from any dataset on demand in one line of R or Python (see the RA guide). The first of those columns is `record_id`, a non-null unique key derived once in the derive stage from whichever provider-issued identifier the record carries. Everything downstream joins on it rather than re-deriving it, and the ladder it is built from is set out with the other columns in Notebook 5..

Of the 71,108 window works, **68,590 (96.5%) carry a working abstract**. Whether an abstract exists and whether it may be sent anywhere are separate questions, and the corpus keeps them in separate columns, because the presence of text is evidence about text and not a grant of rights. `abstract_open_available` is true for **52,196** window works and records only that a usable OpenAlex string is held – many of those strings are unlicensed Microsoft Academic Graph inheritances attached to closed-access works, and they are the ones being withdrawn on copyright grounds (section 1.3). The warrant for external transmission is instead the work's own licence, fetched per work by `collect/openalex-licences.py` and recorded in `OA_licence`. `abstract_externally_sendable` is true only where a usable OpenAlex string is held *and* that licence carries an explicit open slug, which is the case for **21,527** window works, a little under a third of the abstracts the corpus holds.

The screen that reads these abstracts applies a slightly stricter floor than the flag does, requiring 200 characters of OpenAlex abstract rather than the ladder's 100, and on that floor the window divides three ways: **21,493** records whose licence permits external coding, **47,097** that hold a usable abstract but no warrant and are coded by an open-weight model run locally, and the **2,518** with no abstract at all. Each abstract is tagged with its source, its licence, whether it was backfilled and, where it came from an open API, its retrieval source and date, so external eligibility and any future redistribution are decided per work rather than by discarding data. The citation graph (2.77M edges) supports the reference miner; it is kept out of the working dataset so the corpus stays lean.

3.12 What the corpus establishes

1. **Coverage holds at scale.** The OpenAlex database contains 98.4–99.1% of what the curated searches return across 2000–2025; it serves as the discovery substrate and citation backbone.
2. **Discovery is multi-channel by necessity.** The topic-union retrieves about an eighth of the curated trust it contains, and that fraction shrinks over time; recall comes from complementary channels merged by DOI.
3. **The classifier's value is real and large.** The OpenAlex search surfaces 17,881 in-scope-dense, low-noise trust works the keyword searches miss; the curated searches contribute the largest blocks of uniquely-relevant recall. Precision is imposed downstream by the trust-sense triage, the document-type detail, and the content screen.
4. **Three passes closed most of the abstract gap, and the middle one corrected the measurement.** The curated by-DOI backfill recovered curated records for 9,719 pull-only works and enriched the corpus with curated keywords and document types; the ladder fix then reclassified 1,417 placeholder strings as the absences they always were, which took measured coverage down; and the open-API sweep supplied 2,843 working abstracts. Coverage stands at 96.5%, with 2,518 works that lack an abstract anywhere.
5. **Holding an abstract and being allowed to send it are different states, and the corpus records both.** A usable OpenAlex string is held for 52,196 window works; 21,527 of them carry a licence that permits external transmission. The screening partition follows the licence: 21,493 records coded externally, 47,097 by a locally run open-weight model, 2,518 from title and metadata alone.

The 2015 audit chose the design; the 2000–2025 corpus confirms it and is the dataset the downstream stages read. Screening has since run over the externally codeable track under a frozen codebook,

and extraction and reduction work from what it selects. Every column is catalogued in the codebook (Notebook 5.).

4. The THEME pipeline

i Status: visual summary

This notebook is a single picture of how the THEME corpus was built, from the raw searches to the analysis dataset and its codebook. It summarises what the preceding chapters describe in prose; every stage shown has run and produced the counts quoted (August 2026 build). Screening, which reads the finished corpus and decides what each record is, appears here only as the handover point at the foot of the diagram: it ran to completion on the licence-warranted track on 8 August 2026, and its codebook, its three coding tracks and its reliability measurement are set out in Notebook 6.. The stages beyond screening – per-paper extraction from full text, the reduction/review samples, the reference-based "gem" miner, and the versioned release – are left out because they are not yet built; each is taken up separately as it is.

4.1 The pipeline at a glance

Three searches are collected and preserved raw, assembled into one deduplicated corpus (with the curated DOI-backfill folded in as metadata), then enriched with the record key and the analytical variables. Two collectors sit alongside the searches and query by identifier rather than by keyword: a per-work licence fetch, which settles which abstracts may leave the machine, and an abstract-retrieval sweep across the open bibliographic APIs, which repairs the abstracts the local coding track would otherwise have had to work from. The design itself was settled by the 2015 audit (section 3.1) and confirmed on the full corpus (Notebook 3.).

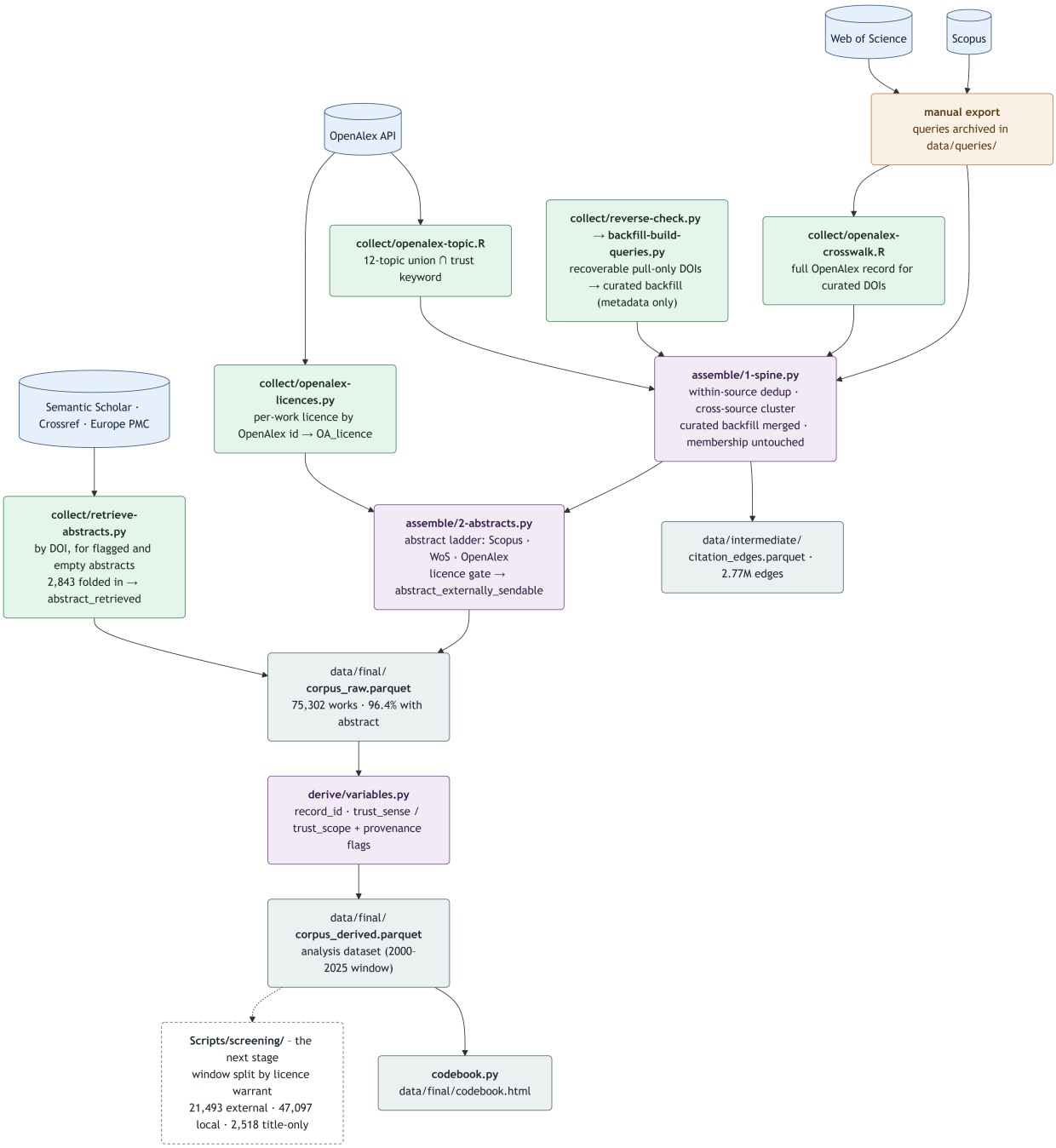

Figure 4.1: The THEME construction pipeline: collection (raw, 1999–2026) ⟶ assembly ⟶ derivation ⟶ the analysis dataset and codebook, with the screening stage that consumes it shown dashed at the foot and described in its own notebook. Colour marks the kind of step: blue a source, green an automated script, amber a manual export, purple an assembly/derivation step, grey an output, dashed the next stage.

4.2 Reading the pipeline

Collection (Stage A) – preserved raw, 1999–2026

Three searches run in parallel, each kept exactly as exported so the raw layer is reproducible. The **OpenAlex topic pull** (`collect/openalex-topic.R`) retrieves the union of twelve curated trust topics intersected with the trust keyword, by API. **Scopus** and **Web of Science** are exported by hand (their interfaces cap programmatic abstract retrieval and their abstracts are licensed for local processing only); the exact queries are archived in `data/queries/` and set out in section 1.4. The **OpenAlex crosswalk** (`collect/openalex-crosswalk.R`) fetches the full OpenAlex record for every curated DOI not already in the topic pull, so the citation graph attaches to the curated works too. Finally, the **curated DOI-backfill** (`collect/reverse-check.py` identifies the recoverable pull-only works; `collect/backfill-build-queries.py` builds the by-DOI queries) recovers curated abstracts, keywords and document types for the only-OpenAlex works that exist in a curated database (section 1.5). 1999 and 2026 are buffer years so 2000/2025 boundary mismatches resolve at merge.

Two further collectors run against the assembled corpus rather than against a query, and both send an identifier out and nothing else. The **per-work licence fetch** (`collect/openalex-licences.py`) asks OpenAlex, for every work in the corpus, what licence is recorded on its primary location and on its best open-access location; the lookup it writes becomes the `OA_licence` column and, through it, the external-API gate.

The **abstract-retrieval sweep** (`collect/retrieve-abstracts.py`) queries Semantic Scholar, Crossref and Europe PMC by DOI for the two groups a local coding run cannot repair as it goes – records holding no usable abstract in any channel, and records whose held abstract fails the screening quality checks (truncated, foreign-language, page furniture, body text, or an abstract belonging to a different publication). 7,454 distinct DOIs were queried; 2,843 replies were usable and flag-clean and became the working abstract, 2,766 of them displacing a defective text and 77 filling an empty one, with `abstract_retrieved` recording which API supplied each and on what date. Retrieved text is marked licensed for local processing, so nothing folded in this way can turn a record into an externally sendable one.

Assembly (Stage B) – one deduplicated corpus

`assemble/1-spine.py` deduplicates each source within itself, clusters the sources into one row per unique work (DOI, then exact normalised title), records which *searches* returned each work (`in_OpenAlex` / `in_WoS` / `in_Scopus`), and attaches the raw metadata from every source. The curated backfill is folded in here as **metadata only** – it fills the `Scopus_*` / `WoS_*` fields and sets the `*_backfilled` flags, but never changes the search-membership flags, so the coverage and added-value results stay intact. The OpenAlex citation graph is split out to `citation_edges.parquet` (kept out of the working corpus). `assemble/2-abstracts.py` then resolves the **abstract ladder** over the held channels (Scopus › WoS › OpenAlex), taking a channel only where its text is usable – at least 100 characters and not a sourced placeholder such as Scopus's `[No abstract available]` – and tags each abstract with the channel it came from, its licence class and whether it was backfilled. The per-work licence joins on here as well: `OA_licence` is attached by OpenAlex id, and `abstract_externally_sendable` combines it with the presence of usable OpenAlex text to give the external-API gate. The retrieval sweep folds its 2,843 abstracts into the same columns afterwards, adding the API and the date in `abstract_retrieved`, so the working abstract can also come from Semantic Scholar, Crossref or Europe PMC. The stage writes `corpus_raw.parquet`: 75,302 works, 96.4% of them carrying a working abstract.

Derivation (Stage C) – the analysis dataset

`derive/variables.py` reads `corpus_raw`, derives `record_id`, applies the trust-sense classifier (`trust_context.py`, Notebook 2.) to compute `trust_sense` / `trust_scope` with their provenance and conflict flags, flags the 2000–2025 analysis window, and writes `corpus_derived.parquet` – the dataset every table in Notebook 3. is computed from. `record_id` is the project-wide key, unique and non-null, written as the first column and built from a ladder of provider-issued identifiers so that it regenerates from the corpus alone: the bare OpenAlex work id where the work has one (70,940 rows), else `doi:` and the DOI (976), else the vendor id already sitting in the open `url` column (`scopus:` 2,780, `wos:` 605, `url:` 1). Every dataset downstream joins on that column rather than minting a key of its own. `codebook.py` renders the derivation registry (`derivations.yaml`) into `data/final/codebook.html`, documenting every column (Notebook 5.). Because derivation does no gathering, it is cheap to re-run whenever the classifier or the corpus changes – as it was after the retrieval sweep, which changed 2,843 abstracts underneath it.

The canonical format is **Parquet** (read directly by the notebooks via R `arrow`); a CSV can be produced from any dataset on demand in one line of R or Python (see the RA guide), so no CSV copies are kept in the repository. Abstracts are retained with their channel, their licence class and their retrieval provenance, so external-API eligibility and any future redistribution are decided per work rather than by discarding data. That decision rests on the work's own licence and never on the presence of text. `abstract_open_available` says only that a usable OpenAlex abstract string is held, and a large share of those strings are unlicensed Microsoft Academic Graph inheritances attached to closed-access works, which is why they are being withdrawn; `abstract_externally_sendable` requires that string *and* an explicit open licence in `OA_licence` – a `cc-*` slug, `cc0` or `public-domain`. A recorded `other-oa`, which names a route to a free copy rather than a grant of rights, does not clear the gate: 1,421 window works carry it and none of them is sendable. 21,527 window works do clear it; screening applies a slightly stricter 200-character floor and sends 21,493 of them for external coding (Notebook 6.), while works holding a usable abstract with no warrant go to a local open-weight model instead, so no licensed text leaves the machine.

5. The THEME codebook

> **i** Status: generated artefact
>
> The codebook is generated from the derivation registry (`Scripts/derivations.yaml`) and the dataset itself by `Scripts/codebook.py`, which writes `data/final/codebook.html`. It documents **every** column of `corpus_derived` – its pipeline stage, the function that builds it, a description, fill rate and an example value – so provenance and documentation live in one place and cannot drift from the data.

`corpus_derived.parquet` is the analysis dataset (one row per unique work). Its columns fall into a small number of families: the **identity** and **membership** fields, the resolved **abstract** with its provenance and licence flags, and the **derived** trust classification are built by the assemble and derive stages and documented one by one in the registry; the bulk of the width is the **raw source metadata** carried verbatim from each source (`OA_*`, `Scopus_*`, `WoS_*`), auto-documented by the codebook generator.

5.1 The record key

The first column of the dataset is `record_id`, the project-wide key, and every dataset built downstream of the corpus joins on it rather than deriving a key of its own. It is computed once, in the derive stage, on a ladder of provider-issued identifiers taken in order: the bare OpenAlex work id (the `https://openalex.org/` prefix stripped) where the work has one, which covers 70,940 of the 75,302 rows; failing that `doi:` and the DOI, 976 rows; failing that the vendor identifier already present in the open `url` column, which gives `scopus:` for 2,780 rows, `wos:` for 605, and `url:` with the landing page as a last resort for one. Every rung is an identifier some provider has already issued, so the key regenerates from the record alone and needs no register maintained beside the data – which is why it was preferred to a project-minted surrogate. It is unique and non-null on all 75,302 rows, and both the pipeline and every build that consumes it fail loudly if that ever ceases to hold.

5.2 The abstract and licence columns

Eight columns describe the working abstract and the terms on which it may be used. `abstract` is the text itself, chosen by the ladder set out in Notebook 4.; `abstract_source` and `abstract_licence` record which channel supplied it; `abstract_backfilled` marks the ones that arrived through the curated DOI-backfill; and `abstract_retrieved` names the open API and the date for the records filled by the August 2026 retrieval sweep, which is blank where the working abstract came from the original feeds. The remaining three carry the rights question. `abstract_open_available` says that a usable OpenAlex abstract string is held, `OA_licence` records the licence OpenAlex holds against the work itself, and `abstract_externally_sendable` is true only where both conditions are met – a usable string *and* an explicit open licence slug.

The registry is emphatic about the distinction those columns draw, because it is easy to lose. Holding an OpenAlex abstract is a claim about text and says nothing about rights: many OpenAlex abstracts are unlicensed Microsoft Academic Graph inheritances attached to closed-access works, and they are being withdrawn as publishers reclassify abstracts as strategic assets. External transmission is warranted by the work's own licence, never by the presence of text, and `abstract_externally_sendable` is the only column that expresses that warrant. The screening partition (Notebook 6.) rests on that warrant, applying a slightly higher abstract-length floor of its own.

5.3 Column families

Family membership is decided by the registry rather than by the column name, because a name can mislead about where a column comes from. `OA_licence` wears the raw-OpenAlex prefix while being an assemble-stage build: it is folded in by `OA_id` from a separate per-work licence fetch (`data/raw/licences/`

`oa_work_licences.parquet`) and belongs with the abstract fields whose external gate it decides. Sorting on the prefix first would file it among the raw OpenAlex search metadata and move a column out of the family that explains it. So the rule below reads the registry first, and falls back on the prefix only for the undocumented long tail of raw source metadata.

Table 5.1: Column families in corpus_derived.

Family	Columns
Web of Science (raw source)	72
Scopus (raw source)	33
OpenAlex (raw source)	28
Identity / year / document type / content	17
Membership + backfill provenance	12
Abstract (resolved + flags)	8
Derived: trust classification	7

The complete column-by-column codebook – every column with its stage, description, fill rate and an example value – is the generated HTML file `data/final/codebook.html`, best read in a browser; it is not reproduced inline in the PDF because of its length.

6. Screening the corpus

i Status: run and measured

Screening sits between the assembled corpus and the extraction pool: each record in the analysis window is read and given two labels under a frozen codebook. The pass over the records that carry an open-licence warrant is complete and its reliability has been measured; the records with no warrant are staged for a locally deployed open-weight model on university HPC. The harness is `Scripts/screening/`, and the populations it draws are partitioned from `data/final/corpus_derived.parquet` by the rule computed below.

The classifier of Notebook 2. reads each record's title, abstract and keyword fields for a lexical cue and assigns a `trust_sense` and a `trust_scope`. It is a prior, cheap and auditable, and it is silent on 43.6% of the window. Screening is the reading that decides what the prior could not: a coder works through the supplied fields and answers two questions about each record, whether it concerns trust in the project's sense and what kind of study it is. The two systems are held in separate columns and neither is computed from the other, so a disagreement between them stays visible and open to review; nothing is resolved silently at merge.

This notebook sets out what is coded, how the corpus was divided between coders on a licence rule, what the completed pass produced and how reliably, what happens to the records too thin to code from an abstract, and what the remaining passes are waiting on.

6.1 Two variables and a frozen codebook

`h_class` asks whether the record is about trust in the project's sense, in five values. `central` means the trust construct is what the paper is about, singled out in the sentences that say what the work does, measures or concludes. `edge-case` means a trust construct is present, measured and reported, but stands level with its neighbours – one of three or more factors carried through the same analysis in the same form. `out-of-scope` means the record is about reliance in a sense the project excludes: a consumer's trust in a brand, a user's trust in a device, a node's trust in a protocol. `fake` means the matched string is a homonym naming no relation of reliance at all – an NHS Trust, a charitable trust, antitrust enforcement – and it counts the homonyms a keyword search on the word inevitably returns. `undetermined` means the supplied fields do not settle the question.

`h_methodology` asks what kind of study the record reports, in seven values: `theoretical`, `qualitative`, `quant`, `mixed`, `review-meta`, and two that record what the metadata fails to establish rather than what the study is. `empirical-uncertain` is for a record that plainly reports an empirical study whose design the supplied fields never name; `uncertain` is for a record that gives the coder nothing to work with. Separating those two matters downstream, because the first is a candidate for full-text review and the second is usually a record artefact.

Each variable carries a written reasoning field, and the reasoning is as much of the deliverable as the label: it is what makes the set usable for calibrating an automatic classifier, and the only means of auditing a close call afterwards. Two diagnostic fields are coded on every row alongside them. `substantiveness` records how much of the record's own claim the trust construct carries, and `evidence_basis` records which of the supplied fields the trust evidence actually sits in – whether the term stands in the abstract, only in a keyword field, or only in the extracted trust snippet.

The codebook that governs all of this is version 3, frozen for production. It holds nine class rules (C1 to C9), thirteen methodology rules (M1 to M13), a rule 0 that requires the coder to name the trustee before coding anything else, and a rule 16 that instructs the coder to flag a configuration the codebook does not settle rather than legislate on it in passing. A further rule, P1, makes retrieval mandatory wherever the pipeline's `abstract_flag` reports the held abstract truncated, foreign-language, page furniture, body text, or belonging to a different publication.

Three calibration waves of 250 records each fixed that version, and the sequence is worth stating because it changed how the project decides when a codebook is finished. The original stopping rule required both a Cohen's kappa above 0.90 on each variable and fewer than three recurring findings from a consistency sweep. Wave 1 met the kappa condition at the first attempt and missed the sweep condition by a factor of fourteen. Wave 2 recoded under the amended version and the sweep returned 43 findings where the first had returned 42, because each rule made more specific brought boundary cases into view that the vaguer rule had never had. Meanwhile the coding itself barely moved: 42 amendments shifted the labels by less than two percentage points above the noise floor of simply running the coder twice. The sweep count could therefore not serve as a stopping rule, since further edges were always available to find. Convergence was redefined on the coding rather than on the auditor, and version 3′s approved scope became *fix the contradictions, flag the gaps*. Its measured effect was as intended: the class coding did not move beyond noise, while the methodology coding moved four to six points in a net-conservative direction, mostly by recording what a record fails to establish instead of inferring a design from an index term.

6.2 Three tracks, divided by licence warrant

Sending an abstract to an external service is a question about rights, not about whether the text is to hand. A large share of the abstracts OpenAlex holds are unlicensed inheritances from the Microsoft Academic Graph, many of them attached to closed-access works, and they are being withdrawn as publishers reclassify abstracts as strategic assets. The project therefore partitions the analysis window on the work's own licence, recorded per work in `OA_licence` and fetched from OpenAlex on 30 July 2026 (section 5.2).

Table 6.1: The 2000–2025 analysis window partitioned into the three screening tracks.

Track	Records
No warrant, abstract held – local model	47,097
Open-licence warrant – coded externally	21,493
No abstract anywhere – title and metadata	2,518
Total (analysis window)	**71,108**

The three tracks are disjoint and cover the window exactly once. The first holds the records whose OpenAlex abstract is long enough to code and whose work carries a licence naming its terms; only these may have their title and abstract sent to an external service, and `abstract_externally_sendable` is the column that says so. The second holds every other record with a usable abstract from any channel, including every abstract repaired or supplied by the August 2026 retrieval sweep, and its coding is assigned to an open-weight model on Newcastle HPC (section 6.6). The third holds the records with no abstract at any index, for which API retrieval is exhausted.

Table 6.2: Licence warrant tiers within the externally coded track. The tiers are nested, so each row reports the records its own licences add and the running total admitted up to and including that tier.

Warrant tier	Records added	Cumulative
strict	15,393	15,393
non-commercial	2,824	18,217
no-derivatives	3,276	21,493

The tiers are nested, so a later decision to tighten or loosen the boundary is a filter on one column rather than a fresh draw. `strict` is cc-by, cc-by-sa, cc0 and public-domain, which bar neither redistribution nor derivatives; `non-commercial` adds cc-by-nc and cc-by-nc-sa; `no-derivatives` adds cc-by-nd and cc-by-nc-nd, and is the tier worth a second look before publication, since a coding label derived from an abstract is arguably not a derivative work of it. One value is deliberately excluded. OpenAlex records `other-oa`

for a work that is open by some route with no recognised licence attached, which is the same evidential position as no licence at all, so its 1,421 window records carry no warrant.

6.3 The open-licence pass

The externally coded pass was dispatched on 2 August 2026 and completed on 8 August: all 21,493 records, in 86 batches of 250, one Claude coding agent per batch, under frozen codebook v3, with P1 retrieval performed inline wherever `abstract_flag` fired.

Batch size was settled by experiment. Wave 2 coded the same 250 records four times, at 25, 50, 100 and 250 records per agent, and found no degradation anywhere in that range: every arm returned a complete and valid file, the quote rate never left 100%, and at the largest batch reasoning length shrank by 0.06 characters per record, which is fifteen characters across the whole batch. Pairwise agreement between arms showed no monotone trend, and the two largest arms agreed with each other more closely than either agreed with the smallest. What did move was self-rated difficulty, which fell steadily with batch size while agreement stayed put – so `difficulty` is not comparable across batch sizes, and a production run has to use one size throughout for that field to be readable at all. The size was then chosen on throughput: at 250 records and sixteen concurrent slots the open-licence pass of 21,493 records takes about four hours of running time.

Durability comes from the result files. Every agent writes its own as it finishes, `screen.py pending` reports exactly which batches have no file, and recovery is re-dispatching that list. The run crossed four session-limit windows – completing 16, 32, 16 and 22 batches in them – and lost nothing at any of the four breaks.

Three checks ran before the results were collected. Completeness was 21,493 of 21,493, every result stamped `codebook_version: v3`, with no invalid files. Quote fidelity across 27,586 quoted spans was 97.8% verbatim in the record, with a further 0.8% quoting text the coder had retrieved under P1 and 2.2% unverified, most of that measurement artefact. P1 compliance was 1,192 of the 1,200 flagged abstracts, or 99% – that check being what stops a coder reading a truncated or misattached abstract and coding from it regardless.

Table 6.3: Labels assigned in the open-licence pass, 21,493 records under codebook v3.

variable	label	records
`h_class`	edge-case	12,059
	central	7,426
	out-of-scope	1,108
	fake	877
	undetermined	23
`h_methodology`	quant	11,448
	qualitative	3,940
	theoretical	3,121
	mixed	1,004
	empirical-uncertain	993
	review-meta	956
	uncertain	31

Quantitative and mixed-methods work together accounts for 12,452 records, which is the population the extraction stage draws from. The 23 `undetermined` and 31 `uncertain` rows are the measure of how rarely a licensed abstract of at least 200 characters leaves a coder with nothing to say.

6.4 Reliability, and the ceiling it sets

Five of the 86 batches were seeded in advance and coded a second time by independent agents forbidden to read the first pass – 1,250 records in all, a blind rather than an adjudicated recode.

Table 6.4: Inter-pass agreement on 1,250 blind-recoded records.

comparison	raw agreement	kappa
`h_class`, five categories	92.3%	0.858
`h_methodology`, seven categories	92.4%	0.885
in-scope against not	98.2%	0.879
quant or mixed against not (the META gate)	96.2%	0.923

Verbatim quote fidelity against title and abstract was 97.86% over the 2,383 quoted spans in that subset.

The collapsed rows say where the full-category disagreement lives. Class disagreement is almost entirely the `central` against `edge-case` boundary, and it is symmetric – 38 records moving up and 32 moving down – which is churn on the claim-sentence test rather than a screening loss, since both values pass into the extraction pool. The decision the funnel actually depends on, whether a record is quantitative or mixed, is the most reliable in the table.

That last figure is also a ceiling, and it governs what the local model may be asked to achieve. The blind pass measures the reference coding against itself, so a model trained on those labels that agreed with them more closely than they agree with each other would be fitting their noise rather than learning the codebook. Two of the acceptance gates in the handover kit were set on that reasoning: quant-or-mixed recall at 0.94 and rule 16 flag emission at 0.78, each just below the reference coder's own measured self-agreement on the same quantity, 0.945 and 0.788. A gate set above that line would fail a perfect clone.

A second reliability strand measures the coding against a human reading rather than against itself. A first human round in June 2026 coded 116 records and returned much weaker agreement with a machine coding of the same records – kappa 0.39 on `h_class` and 0.50 on `h_methodology` – under a protocol that predates codebook v3, and the disagreements it exposed are a large part of what v3 was written from. A second round is prepared: 400 records drawn blind from the completed open-licence pass and stratified so that the thin categories carry enough cases to say something about, coded under the frozen codebook without sight of the machine labels. Its human-machine agreement will be reported beside the machine-machine figures above rather than in place of them, since the two answer different questions.

6.5 The title-only pass

The 2,518 records with no abstract at any index were not deferred. Book chapters, editorials and front matter dominate the group, and of the 1,736 that carry a DOI, the August 2026 retrieval sweep returned a usable abstract for none. The sweep did fill 77 records that had held nothing, and those left the group for the local-model track, taking it from 2,595 to its present size; for the residue that remains, API retrieval is exhausted, so waiting for text was not going to change their position. They were coded from title and metadata alone in eleven batches through the same harness, under codebook v3 as applied by a written title-only protocol.

The labels are weak by design, and two fields put that weakness in the data itself. `evidence_basis` records which supplied field the trust evidence sits in, which for this population can never be the abstract. `p1_status` carries a byte-exact marker on every row – "P1 check not performed; no external retrieval available" – because the coding has to be reproducible from the batch file alone. The pass is stamped `v3-titleonly` and is never mixed into the open-licence deliverable, since the reliability figures above describe abstract-based coding and nothing else.

Table 6.5: Labels assigned in the title-only pass, 2,518 records.

variable	label	records
h_class	central	1,484
	fake	620
	edge-case	364
	undetermined	38
	out-of-scope	12
h_methodology	uncertain	1,230
	empirical-uncertain	733
	theoretical	390
	review-meta	156
	quant	7
	qualitative	2

The shape is the one the protocol predicted. Codes that require a named design are nearly unreachable without an abstract, so `quant` is reached on seven records and the two uncertainty codes take 78% of the pass between them. The high `fake` share is the homonym harvest of a population dominated by book chapters and front matter. Quote fidelity, measured with the pairing-aware span extractor over the full record, was 1,821 spans with none unverified.

What to do with these records is the open question. Three candidate tests would route them into the full-text extraction stage, and they select very different populations: the existing quant-or-mixed gate takes 7 records, adding `empirical-uncertain` as this pass's stand-in for a design that full text would resolve takes 578, and taking everything not positively excluded takes 1,484. The rest become candidates for documented exclusion, which is a decision the project intends to record rather than let happen by attrition.

6.6 The local-model track

The 47,097 records with a usable abstract and no licence warrant are assigned to an open-weight model on Newcastle HPC, and every one of them now holds a working abstract: the retrieval sweep repaired 2,766 defective texts and supplied 77 missing ones from Semantic Scholar, Crossref and Europe PMC. That mattered for this track specifically, because the local run codes from batch files on nodes with no network access, so an abstract too damaged to read cannot be repaired at coding time.

The handover kit is self-contained: standing the model up needs nothing from the compendium repository. It carries the frozen codebook condensed into a system prompt a 32B-class model can hold, worked exemplars, a JSON schema for constrained decoding, the generation client, and a scoring harness. The training splits reserve the 1,250 blind-pass records entirely, so they can serve as the evaluation reference, and the split is reproducible from a recorded seed. The kit also ships a self-test in which the harness scores the blind second pass against the first and reproduces the project's kappa figures to three decimals – the point being that whoever runs it can check the measuring instrument before trusting a number it gives them about a new model.

The sequence is baseline first. The model is evaluated on the reserved batches as it comes, against the gates described above, before any fine-tuning is considered; a model that clears them untuned makes the training splits unnecessary.

6.7 The screening datasets

Each stage of the funnel has a `_raw` dataset holding everything that enters it and a `_prod` dataset holding what leaves, and each stage's `_prod` is the next stage's input. A track name inserted between the two

marks a partial build covering one coding track only, so the passes stay separable in the methods reporting whatever is later done with their union.

Table 6.6: The THEME screening datasets and their state, August 2026.

dataset	rows	what it holds	state
`theme_raw`	75,302	every corpus record, open columns only, keyed on `record_id`	built
`theme_claude_prod`	21,493	the open-licence track under codebook v3	built
`theme_titleonly_prod`	2,518	the abstract-less records, weak labels	built
`theme_local_prod`	47,097	the no-warrant track, coded on HPC	pending the local run
`theme_prod`	–	both tracks combined and pruned on `h_class` and `trust_scope`	pending
`meta_raw`	11,654	the quantitative candidates carried forward	built, one track only

`theme_raw` reads only the columns that carry no redistribution restriction: every field prefixed `Scopus_` or `WoS_` is left behind, and `record_id` (section 5.1) sits in first position as the key everything else joins on.

6.8 Into META

`meta_raw` is the candidate pool for full-text extraction, and three filters build it from the open-licence codings in order.

Table 6.7: The filter chain from the screened corpus into the extraction pool.

filter	rule	removed	remaining
1	`h_class` must be central or edge-case	2,008	19,485
2	`h_methodology` must be quant or mixed	7,735	11,750
3	`trust_scope` must not be out-of-scope, unless `h_class` is central	96	11,654

Filter 2 is the gate the funnel depends on, and it is deliberately quant *or* mixed: mixed-methods work reports numerical results and belongs in the extraction pool. It is also the most reliable decision the coding makes, at kappa 0.923.

Filter 3 carries two rulings worth stating. Records the lexical classifier left `undetermined` are kept, because the classifier failing to resolve a sense says nothing about the record – only an explicit out-of-

scope verdict removes one. And a record a coder judged `central` stays whatever the triage said, since the coder read the text and the triage matched words; the 124 records kept that way retain `trust_scope` of `out-of-scope` in the data, so they remain directly filterable for the manual review intended later.

The resulting pool is 11,654 records: 5,200 central and 6,454 edge-case, 10,720 quant and 934 mixed, with 7,875 in scope on the lexical sense, 3,655 undetermined and the 124 kept on the central override. It is a candidate pool rather than a work list. The criteria that narrow it to the set whose full texts are actually sourced are still being designed, and because the pool is built from one coding track it will be rebuilt from the union of both once the local run lands.

The parsing question at the far end is settled. A pilot on 24 June 2026 put 21 PDFs through GROBID v0.9.0: all 21 parsed, and 923 trust-bearing sentences were extracted from them. The method bank that selects those sentences needs precision tuning before it runs at scale. The extraction field list is drafted but not yet settled, and settling it is what locks the schema the extraction stage decodes against, so it has to come before any of `meta_prod` can be built.

6.9 What is pending

Five things are outstanding.

The **local production run** has to happen: the model has to clear the acceptance gates on the reserved evaluation batches, then code the 47,097 records, after which `theme_local_prod` and `theme_prod` can be built and `meta_raw` rebuilt from the union of the two tracks.

The **routing test for the title-coded records** has to be chosen from the three candidates, and the records it does not select recorded as documented exclusions.

The **extraction field list** has to be fixed, since the schema for the extraction stage cannot be built against a moving target, and the **HPC project request** has to be filed before any of that compute can be scheduled.

Finally, `meta_raw` will need **full texts**. Everything above operates on bibliographic metadata; the extraction stage is the first that needs the papers themselves, and PDF sourcing against the pool has not begun.

II

META – the meta-analytical dataset

III METHOD – the methodological dataset

IV References

Alonso-Álvarez, P. & Eck, N.J. van (2025) "Coverage and Metadata Completeness and Accuracy of African Research Publications in OpenAlex: A Comparative Analysis," *Quantitative Science Studies,* 6pp. 1336–1357.

Culbert, J.H., Hobert, A., Jahn, N., Haupka, N., Schmidt, M., Donner, P. & Mayr, P. (2025) "Reference Coverage Analysis of OpenAlex Compared to Web of Science and Scopus," *Scientometrics,* 130(4), pp. 2475–2492.

Eck, N.J. van (2024) *Classification of Research Publications Based on Data from OpenAlex (Version 2023nov).*

Eck, N.J. van & Waltman, L. (2024) *An Open Approach for Classifying Research Publications.* [Online] [online]. Available from: https://www.leidenmadtrics.nl/articles/an-open-approach-for-classifying-research-publications.

Initiative for Open Abstracts (2020) *Initiative for Open Abstracts (I4OA).* [Online] [online]. Available from: https://i4oa.org/.

Kim, S., Holst, V. & Ginis, V. (2026) "One in Eight OpenAlex Abstracts Has Integrity Issues," *arXiv preprint,*

Priem, J., Piwowar, H. & Orr, R. (2022) "OpenAlex: A Fully-Open Index of Scholarly Works, Authors, Venues, Institutions, and Concepts," *arXiv preprint,*

Stansfield, C., Dehdarirad, H., Thomas, J., Mathew, S. & O'Mara-Eves, A. (2025) "Analysing the Utility of OpenAlex to Identify Studies for Systematic Reviews: Methods and a Case Study," *(preprint),*

Tay, A. (2025) *The Petrol Tank for AI Discovery Might Be Running Dry as Publishers Close Access to Scholarly Content Such as Abstracts Due to AI Incentives.* [Online] [online]. Available from: https://aarontay.substack.com/p/the-petrol-tank-for-ai-discovery.

Traag, V.A., Waltman, L. & Eck, N.J. van (2019) "From Louvain to Leiden: Guaranteeing Well-Connected Communities," *Scientific Reports,* 9p. 5233.

Waltman, L. & Eck, N.J. van (2012) "A New Methodology for Constructing a Publication-Level Classification System of Science," *Journal of the American Society for Information Science and Technology,* 63(12), pp. 2378–2392.